Implementación de autenticación básica

I have an **ASP.NET** **Core Web API backend** project running in a Docker container plus a sql server running in another docker container. It's all for a booking app. I've already successfully implemented CRUD endpoints in Controllers writing the corresponding code using Visual Studio IDE. And I've also successfully tested all "GET all" and POST endpoints using both Swagger and Postman.

The structure of my project is the following:

reservas-app folder is the root of it all and contains four folders: 'backend', 'deployment', 'frontend' and 'cronservice'.

'cronservice' folder currently only contains .git folder, .gitignore file and README.md file.

'frontend' folder contains all the basics for an Angular frontend, but I have not implemented anything that the user can see or click on, I probably have just the typical Angular frontend template. To be more specific, my 'frontend' folder currently contains .git folder, .vscode folder, node_modules folder, public folder, src folder, .editorconfig file, .gitignore file, angular.json file, Dockerfile, package.json file, package-lock.json file, README.md file, tsconfig.app.json file, tsconfig.json file and tsconfig.spec.json file.

To the contrary, I've already done a lot of coding in my backend project, for instance the code for implementing the CRUD endpoints. What I have inside this backend folder is currently the following:

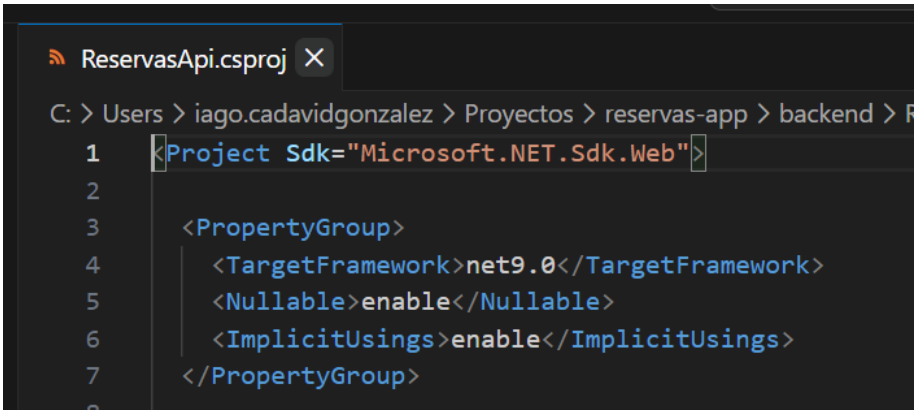
.git folder, .github folder, .vs folder, .vscode folder, ReservasApi folder, ReservasApi.Tests folder, .dockerignore file, .gitignore file, backend.sln file and Docker file.

Inside ReservasApi folder I have the following:

bin folder, Controller folder (CRUD endpoints files and code), Data folder, Eventos folder (for Kafka events, but all Kafka code is currently commented so it's like it does not exist), Migrations folder, Modelos folder (models for tables of bd), obj folder, Properties folder, appsettings.Development.json file, appsettings.json file, Program.cs file, ReservasApi.csproj file, ReservasApi.csproj.user file and ReservasApi.http file.

Also worth mentioning my backend code is c sharp i.e. c# code.

Now I've been told to "implement basic authentication", I guess they are referring to user register plus user login. Please **guide** me through the process of "**implementing basic authentication**".



Regarding "Your backend is .NET 6–8 (the process is the same)." in ReservasApi.csproj what I have is <TargetFramework>net9.0</TargetFramework>

Great — **.NET 9.0** works perfectly fine.
All instructions I gave are fully compatible with `.NET 9`, and there is nothing you need to change in the authentication steps because:

✔ ASP.NET Core identity & JWT authentication work the same in .NET 6, 7, 8, and 9

The differences between versions do *not* affect:

- JWT authentication
- Middleware pipeline (`UseAuthentication`, `UseAuthorization`)
- Controller attributes (`[Authorize]`, `[AllowAnonymous]`)
- Dependency injection
- Entity Framework Core authentication logic

So you can follow the implementation exactly as described.

🎯 Just verify one thing in Program.cs (if you're using minimal hosting)

Since .NET 6 Microsoft moved to the *minimal hosting model* and .NET 9 uses it by default.

Your Program.cs likely looks like:

csharp

```csharp
var builder = WebApplication.CreateBuilder(args);

// Add services here

var app = builder.Build();

// Configure middleware here

app.Run();
```

En efecto mi Program.cs tiene esas 3 líneas y entre las primeras dos tengo lo de add services:

```csharp
var builder = WebApplication.CreateBuilder(args); // esto ya ven�a por defecto.

// 1. Add services to the container.

// Add controllers (this enables the [ApiController] controllers)
builder.Services.AddControllers(); //.AddOpenApi();
    /*
    .AddJsonOptions(opciones => // esto lo a�ado con la intenci�n de que get api reservas
    {
        opciones.JsonSerializerOptions.ReferenceHandler = System.Text.Json.Serialization.Re
        opciones.JsonSerializerOptions.WriteIndented = true;
    });
    */

// Add EF Core and configure SQL Server using the connection string from appsettings.json
builder.Services.AddDbContext<AppDbContext>(opciones =>
    opciones.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection"))
);

// Add Swagger/OpenAPI support for testing your endpoints easily
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen(); // requires installation of the Swagger (Swashbuckle) Nu

// Register the Kafka producer in DI, para ello a�ado esta l�nea antes de var app = build
//builder.Services.AddSingleton<IServicioProductorKafka, ProductorKafka>(); // ProductorKaf
// COMENTO LA L�NEA ANTERIOR DE CARA A EMPEZAR CON TESTS UNITARIOS IGNORAR TODO LO DE KAFK

var app = builder.Build(); // ya viene por defecto.
```

Entre la 2nda y 3era líneas tengo esto:

```
32  var app = builder.Build(); // ya viene por defecto.
33  
34  // 2. Configure the HTTP request pipeline.
35  if (app.Environment.IsDevelopment()) // esto ya ven?a por defecto
36  {
37      //app.MapOpenApi(); este es el ?nico que ven?a por defecto.
38      // Enable Swagger UI only in development :
39      app.UseSwagger(); // esto lo puse yo
40      app.UseSwaggerUI(); // esto lo puse yo
41  }
42
43  app.UseHttpsRedirection(); // ya ven?a por defecto.
44
45  app.UseAuthorization(); // lo puse yo.
46
47  // Map controller routes (this enables your /api/... endpoints)
48  app.MapControllers(); // lo puse yo.
49
50  app.Run(); // lo puse yo.
```

You only need to make sure:

csharp

```
// BEFORE app.Build()
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(/* ... */);

builder.Services.AddAuthorization();
builder.Services.AddScoped<AuthService>();

// AFTER app.UseRouting()
app.UseAuthentication();
app.UseAuthorization();
```

Página siguiente

In my backend Program.cs I don't have the following :

builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(/* ... */);

builder.Services.AddAuthorization();
builder.Services.AddScoped<AuthService>();

Should I add it before var app = builder.Build() ?

Yes — **exactly**.
In ASP.NET Core (including .NET 9), all **service registrations** must be added **before**:

csharp Copy code

```csharp
var app = builder.Build();
```

So you should insert the authentication setup **above** app.Build(), along with your other `builder.Services.*` lines.

NEXT PAGE

When I type in Program.cs (using Visual Studio Code) builder.Services.AddAuth it doesn't autocomplete the rest, which for instance does happen with builder.Services.AddSwaggerGen()

Ah — that's a common **VS Code IntelliSense issue** and also sometimes a **missing using directive**. Unlike `AddSwaggerGen()`, `AddAuthentication()` is an extension method that lives in **Microsoft.AspNetCore.Authentication** namespaces. If the compiler/IntelliSense doesn't know about it, you won't get autocomplete.

Here's how to fix it:

---

1 Add the proper `using` statements at the top of Program.cs

csharp Copy code

```csharp
using Microsoft.AspNetCore.Authentication.JwtBearer;
using Microsoft.IdentityModel.Tokens;
using System.Text;
```

- `Microsoft.AspNetCore.Authentication.JwtBearer` → contains `AddAuthentication()` and `AddJwtBearer()`.
- `Microsoft.IdentityModel.Tokens` → needed for `SymmetricSecurityKey` and `TokenValidationParameters`.
- `System.Text` → needed for `Encoding.UTF8.GetBytes(...)`.

---

2 Make sure the NuGet package is installed

You need `Microsoft.AspNetCore.Authentication.JwtBearer`. In your **backend folder** run:

bash Copy code

```bash
dotnet add package Microsoft.AspNetCore.Authentication.JwtBearer
```

> After adding it, restart VS Code so IntelliSense refreshes.

---

3 Check IntelliSense again

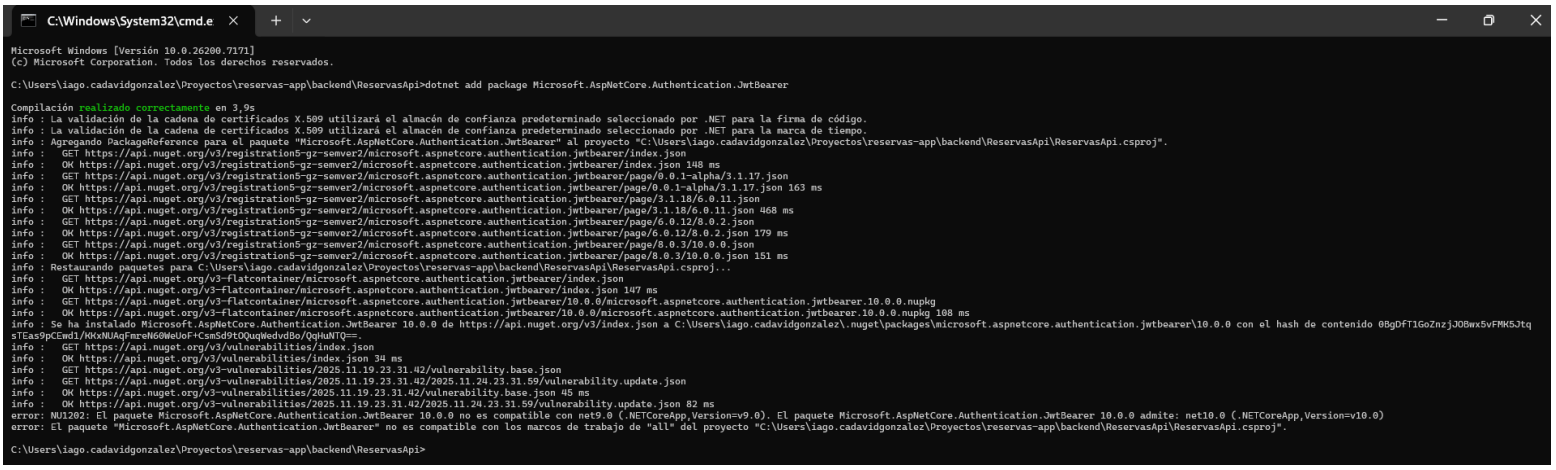
Now typing:

csharp Copy code

```csharp
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
```

En mi ReservasApi.csproj no hay ningún PackageReference con Microsoft.AspNetCore.Authentication.JwtBearer, así que desde ReservasApi folder (padre del csproj file) abro cmd y ejecuto:

dotnet add package Microsoft.AspNetCore.Authentication.JwtBearer para añadir dicho PackageReference al csproj file.



F

I've just opened cmd from ReservasApi folder (i.e. parent folder of ReservasApi.csproj file) and ran dotnet add package Microsoft.AspNetCore.Authentication.JwtBearer, but gott error: The package Microsoft.AspNetCore.Authentication.JwtBearer 10.0.0 is not compatible with net9.0

Ah! That error explains everything. In **.NET 9**, the versioning of NuGet packages changed slightly. The latest `Microsoft.AspNetCore.Authentication.JwtBearer` package **10.0.0** targets .NET 10, so it's **not compatible with net9.0**.

In .NET 6–9, **you don't need to install a separate package** — JWT authentication is **already included** in the shared framework (`Microsoft.AspNetCore.App`).

✅ What this means for your project

1. **Do not run** `dotnet add package Microsoft.AspNetCore.Authentication.JwtBearer` **for .NET 9.**
2. Just make sure you have these using statements in `Program.cs`:

csharp — Copy code

```
using Microsoft.AspNetCore.Authentication.JwtBearer;
using Microsoft.IdentityModel.Tokens;
using System.Text;
```

3. The `builder.Services.AddAuthentication()` and `.AddJwtBearer(...)` code works **without any extra NuGet package** because it's part of the ASP.NET Core shared framework.

```
// Insert the authentication setup above "app.Build()", along with your other builder.Services.*
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme).AddJwtBearer(/*...*/);
builder.Services.AddAuthorization();
builder.Services.AddScoped<AuthService>();
```

Lo tuve que poner “a mano” entonces le dije a chatgpt que me lo chequeara, me viene a decir que está bien pero que debo reemplazar /*...*/ por algo más concreto:

1️⃣

`builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme).AddJwtBearer(/*...*/);`

✅ **Correct structure**, but:

- The `/*...*/` placeholder needs to be replaced with a lambda that configures `TokenValidationParameters`, e.g.:

csharp — Copy code

```
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(options =>
    {
        options.TokenValidationParameters = new TokenValidationParameters
        {
            ValidateIssuer = false,
            ValidateAudience = false,
            ValidateIssuerSigningKey = true,
            IssuerSigningKey = new SymmetricSecurityKey(
                Encoding.UTF8.GetBytes(builder.Configuration["Jwt:Key"])
            )
        };
    });
```

This is **required** for JWT authentication to actually work.

NEXT PAGE

2 `builder.Services.AddAuthorization();`

✅ Correct.

- This registers the Authorization services in the DI container.
- Needed for `[Authorize]` attributes to function.

---

3 `builder.Services.AddScoped<AuthService>();`

✅ Correct.

- Registers your custom `AuthService` so it can be injected into controllers.
- `Scoped` is the right lifetime for services that access DbContext.

⚠️ **Important Notes**

1. Make sure all **using statements** are present at the top of Program.cs:

csharp

```
using Microsoft.AspNetCore.Authentication.JwtBearer;
using Microsoft.IdentityModel.Tokens;
using System.Text;
```

Ahora mismo tengo además de los 3 using, esto:

```
// Insert the authentication setup above "app.Build()", along with your other builder.Services.*
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(opciones =>
    {
        opciones.TokenValidationParameters = new TokenValidationParameters // me autocompletó el primer TVP, no el segundo.
        {
            ValidateIssuer = false,
            ValidateAudience = false,
            ValidateIssuerSigningKey = true,
            IssuerSigningKey = new SymmetricSecurityKey(
                Encoding.UTF8.GetBytes(builder.Configuration["Jwt:Key"])
            )
        };
    }); // lo que viene justo después de AddJwtBearer es una lambda that configures TokenValidationParameters.
builder.Services.AddAuthorization();
builder.Services.AddScoped<AuthService>();

// in ASP.NET Core (including .NET 9), all service registrations must be added before:
var app = builder.Build(); // ya viene por defecto.
```

Según chatgpt my lambda code está bien y es válido para mi caso concreto, el cual se lo expliqué al detalle.

- The lambda goes **inside** `AddAuthentication(...).AddJwtBearer(...)` **before** `builder.Build()`.
- After `app = builder.Build()`, call:

```csharp
app.UseAuthentication();
app.UseAuthorization();
```

✅ Conclusion

- Your code is **functionally correct**.

Anteriormente esto lo explicó así:

✅ PART 6 — Enable JWT Authentication in `Program.cs`

Add at top:

```csharp
using Microsoft.AspNetCore.Authentication.JwtBearer;
using Microsoft.IdentityModel.Tokens;
using System.Text;
```

Add **before** `builder.Build()`:

```csharp
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(options =>
    {
        options.TokenValidationParameters = new TokenValidationParameters
        {
            ValidateIssuer = false,
            ValidateAudience = false,
            ValidateIssuerSigningKey = true,
            IssuerSigningKey = new SymmetricSecurityKey(
                Encoding.UTF8.GetBytes(builder.Configuration["Jwt:Key"])
            )
        };
    });

builder.Services.AddScoped<AuthService>();
```

Add **after** `app.UseRouting()`:

```csharp
app.UseAuthentication();
app.UseAuthorization();
```

- **Full corrected Program.cs for your backend**

csharp

```csharp
using Microsoft.EntityFrameworkCore;
using ReservasApi.Data;
using Microsoft.AspNetCore.Authentication.JwtBearer;
using Microsoft.IdentityModel.Tokens;
using System.Text;

var builder = WebApplication.CreateBuilder(args);

// ----------------------------
// 1. Add services
// ----------------------------

builder.Services.AddControllers();

builder.Services.AddDbContext<AppDbContext>(opciones =>
    opciones.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection"))
);

builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

// JWT Authentication
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(opciones =>
    {
        opciones.TokenValidationParameters = new TokenValidationParameters
        {
            ValidateIssuer = false,
            ValidateAudience = false,
            ValidateIssuerSigningKey = true,
            IssuerSigningKey = new SymmetricSecurityKey(
                Encoding.UTF8.GetBytes(builder.Configuration["Jwt:Key"])
            )
        };
    });

builder.Services.AddAuthorization();
builder.Services.AddScoped<AuthService>();

// ----------------------------
// 2. Build the app
// ----------------------------

var app = builder.Build();

// ----------------------------
// 3. Configure middleware
// ----------------------------

if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();

// Authentication must come before Authorization
app.UseAuthentication();
app.UseAuthorization();

// Map controllers (your /api/... endpoints)
app.MapControllers();

app.Run();
```

Lo hago yo:

Program.cs | ReservasApi.csproj

C: > Users > iago.cadavidgonzalez > Proyectos > reservas-app > backend > ReservasApi > Program.cs > Program > <top-level-statements-entry-point>

```
1   using Microsoft.EntityFrameworkCore; // aparece al pone UseSqlServer
2   using ReservasApi.Data;
3
4   // using ReservasApi.Eventos; // puse esta l□nea al poner con "Register the Kafka producer in DI (Dependency Injection)"
5
6   // para autenticación:
7   using Microsoft.AspNetCore.Authentication.JwtBearer; // no intellisense autocompletion
8   using Microsoft.IdentityModel.Tokens; // no intellisense autocompletion
9   using System.Text; // intellisense me acierta el Text.
10
11  var builder = WebApplication.CreateBuilder(args); // esto ya ven□a por defecto.
12
13  // 1. Add services to the container.
14
15  // Add controllers (this enables the [ApiController] controllers)
16  builder.Services.AddControllers(); //.AddOpenApi();
17      /*
18      .AddJsonOptions(opciones => // esto lo a□ado con la intenci□n de que get api reserva
19      {
20          opciones.JsonSerializerOptions.ReferenceHandler = System.Text.Json.Serialization.R
21          opciones.JsonSerializerOptions.WriteIndented = true;
22      });
23      */
24
25  // Add EF Core and configure SQL Server using the connection string from appsettings.json
26  builder.Services.AddDbContext<AppDbContext>(opciones =>
27      opciones.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection"))
28  );
29
30  // Add Swagger/OpenAPI support for testing your endpoints easily
31  builder.Services.AddEndpointsApiExplorer();
32  builder.Services.AddSwaggerGen(); // requires installation of the Swagger (Swashbuckle) Nu
33
34  // Register the Kafka producer in DI, para ello a□ado esta l□nea antes de var app = buil
35  //builder.Services.AddSingleton<IServicioProductorKafka, ProductorKafka>(); // ProductorKa
36  // COMENTO LA L□NEA ANTERIOR DE CARA A EMPEZAR CON TESTS UNITARIOS IGNORAR TODO LO DE KAF
37
```

Sigue en next page

```
38  // Insert the authentication setup above "app.Build()", along with your other builder.Services.*
39  // JWT Authentication:
40  builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
41      .AddJwtBearer(opciones =>
42      {
43          opciones.TokenValidationParameters = new TokenValidationParameters // me autocompletó el primer TVP, no el segundo.
44          {
45              ValidateIssuer = false,
46              ValidateAudience = false,
47              ValidateIssuerSigningKey = true,
48              IssuerSigningKey = new SymmetricSecurityKey(
49                  Encoding.UTF8.GetBytes(builder.Configuration["Jwt:Key"])
50              )
51          };
52      }); // lo que viene justo después de AddJwtBearer es una lambda that configures TokenValidationParameters.
53  builder.Services.AddAuthorization();
54  builder.Services.AddScoped<AuthService>();
55
```

```
56  // 2. Build the app
57  // in ASP.NET Core (including .NET 9), all service registrations must be added before:
58  var app = builder.Build(); // ya viene por defecto. Esta línea es toda la sección 2. Build the app
59
60  // 2. Configure the HTTP request pipeline.
61  // A 25 de nov 2025 (al estar con lo de la autenticación), chatgpt a esto le llama 3. Configure middleware
62  if (app.Environment.IsDevelopment()) // esto ya ven?a por defecto
63  {
64      //app.MapOpenApi(); este es el ?nico que ven?a por defecto.
65      // Enable Swagger UI only in development :
66      app.UseSwagger(); // esto lo puse yo
67      app.UseSwaggerUI(); // esto lo puse yo
68  }
69
70  app.UseHttpsRedirection(); // ya ven?a por defecto.
71
72  // Authentication must come before Authorization:
73  app.UseAuthentication();
74  app.UseAuthorization(); // lo puse yo.
75
76  // Map controller routes (this enables your /api/... endpoints)
77  app.MapControllers(); // lo puse yo.
78
79  app.Run(); // lo puse yo.
```

Regarding "Full corrected Program.cs for your backend", you did not write app.UseRouting()

Exactly — in your **.NET 9 minimal hosting setup**, you do **not need** `app.UseRouting()`. Here's why:

Añado el Jwt con la super secret key that is at least 32 chars long (conté que la que puse tiene 33 chars):

Program.cs | appsettings.json | ReservasApi.csproj

C: > Users > iago.cadavidgonzalez > Proyectos > reservas-app > backend > ReservasApi > appsettings.json > Jwt > Key

```
{
  "ConnectionStrings":{
    "DefaultConnection": "Server=db;Database=ReservasDb;User Id=sa;Password=docKer!8;TrustServerCertificate=True;"
  },
  "Jwt": {
    "Key": "aP9d73D1jD0m3Ok1Lma17uiWe05vNfg57"
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*"
}
```

Y eso es este paso:

✅ PART 5 — Add JWT configuration in `appsettings.json`

json

```
"Jwt": {
  "Key": "some-super-secret-key-at-least-32-bytes-long"
}
```

In production, use environment variables or Docker secrets.

Otra cosa que según chatgpt debo tocar para lograr la implementación de autenticación básica es mi modelo de tabla de usuarios, que actualmente es así:

NEXT PAGE

```csharp
using System.ComponentModel.DataAnnotations;

namespace ReservasApi.Modelos
{
    7 references
    public class Usuario
    {
        [Key]
        5 references
        public string Dni { get; set; } = string.Empty; // el dni es la primary key referenciada por fk en tabla de reservas

        2 references
        public string NombreCompleto { get; set; } = string.Empty;
        0 references
        public string Email { get; set; } = string.Empty;
        0 references
        public string Telefono { get; set; } = string.Empty;
        0 references
        public string Domicilio { get; set; } = string.Empty;

        // fechas:
        0 references
        public DateOnly FechaNacimiento { get; set; }
        0 references
        public DateTime FechaRegistro { get; set; }

        // contraseña:
        [Required] // makes the property required, so that it cannot be null or omitted when inserting/updating.
        0 references
        public string Contrasinal { get; set; } = string.Empty; // this is "the property declaration", above it pones la 'Required' keyword.

        // propiedad de navegación: un usuario realiza entre 0 y varias reservas
        0 references
        public ICollection<Reserva>? Reservas { get; set; } // el cero lo represento con ? y el varios con ICollection.
    } // final de public class Usuario
}
```

Sobre ese código, dice chatgpt lo siguiente: “your current ‘Usuario’ model mostly represents a **business user**” (with DNI, name, email, phone, address, reservastions, etc.), which is fine for your booking app. But for **secure authentication**, it’s not enough.

1 Problem with your current model

```csharp
[Required]
public string Contrasinal { get; set; } = string.Empty;
```

- Right now you’re storing the **raw password** (`Contrasinal`) as a string.
- **Security risk**: storing plain text passwords is **very unsafe**.
- For JWT-based authentication, you need:
  - **Password hash** (`byte[]`)
  - **Password salt** (`byte[]`)
- This allows you to **verify passwords without storing the raw password.** which is a security risk.

NEXT PAGE

2️⃣ How to modify your `Usuario` model for authentication

You have two options:

Option A — Separate authentication properties

Keep your business fields (DNI, Name, Email, etc.) and **add**:

```csharp
[Required]
public byte[] PasswordHash { get; set; }

[Required]
public byte[] PasswordSalt { get; set; }
```

- Remove or ignore the plain `Contrasinal` property.
- Use `PasswordHash` and `PasswordSalt` in your login/registration logic.

```
/*
// contraseña as a raw string NO es segura:
[Required] // makes the property required, so that it cannot be null or omitted when inserting/updating.
public string Contrasinal { get; set; } = string.Empty; // this is "the property declaration", above it pones la 'Requir
*/

// Reemplazo la contraseña as raw string (no segura) por pwhash y pwsalt:
[Required]
0 references
public byte[] PasswordHash { get; set; }
[Required]
0 references
public byte[] PasswordSalt { get; set; }
```

Ahora bien, esos cambios supongo que me perturbarán los testeos con swagger y postman, ya que supongo que tendré que acordemente cambiar alguna otra cosilla por ahí.

NEXT PAGE

Lo que acabo de hacer es esto:

3 Recommended change (simplest)

Modify your model like this:

```csharp
[Required]
public byte[] PasswordHash { get; set; }

[Required]
public byte[] PasswordSalt { get; set; }
```

- Remove this line:

```csharp
public string Contrasinal { get; set; } = string.Empty;
```

- Your existing fields (DNI, NombreCompleto, Email, etc.) can stay — they will be returned in APIs after login.

Y justo después deso dice esto (sin saber mi código actual al respecto):

4 Update DbContext & database

```csharp
public DbSet<Usuario> Usuarios { get; set; }
```

Then:

```bash
dotnet ef migrations add AddUsuarioAuth
dotnet ef database update
```

=

✅ PART 2 — Add DbSet in your DbContext

Open **Data/ReservasContext.cs** and add:

```csharp
public DbSet<Usuario> Usuarios { get; set; }
```

Then run migration:

```bash
dotnet ef migrations add AddUsuarios
dotnet ef database update
```

(Dockerized SQL Server works fine, connection string must match.)

Regarding "Update DbContext" know that I already have a folder named Data (which is inside the folder named 'ReservasApi', which is inside the folder named 'backend'), and this folder named Data has just one thing inside, which is the a file named AppDbContext.cs, which has the following relevant code (with relevant I mean regarding Usuario):

```
public class AppDbContext : DbContext
{
    public DbSet<Usuario> Usuarios {get; set;}
}
```

Perfect — that's exactly what you need in your `AppDbContext` for Entity Framework to know about your `Usuario` table. A few notes and next steps regarding **authentication**:

1️⃣ Your DbContext is already correctly set up

csharp Copy code

```
public class AppDbContext : DbContext
{
    public DbSet<Usuario> Usuarios { get; set; }
}
```

- EF Core now knows to create a `Usuarios` table corresponding to your `Usuario` model.
- Once you add `PasswordHash` and `PasswordSalt` properties to `Usuario`, EF will detect the new columns in the next migration.

Lo de add PasswordHash y PasswordSalt (y a cambio quitar la raw string pw) ya lo hice hace un rato.

NEXT PAGE

2 Next step: Update `Usuario` for authentication

As we discussed, modify your `Usuario` model to replace the plain `Contrasinal` property with:

```csharp
[Required]
public byte[] PasswordHash { get; set; }

[Required]
public byte[] PasswordSalt { get; set; }
```

- Keep all your other business fields (DNI, NombreCompleto, etc.) — they can coexist.

---

3 Create a new EF migration

After changing the `Usuario` model:

1. Open a terminal in the `ReservasApi` **folder** (where `ReservasApi.csproj` lives).
2. Run:

```bash
dotnet ef migrations add AddUsuarioAuth
dotnet ef database update
```

- This will:
  - Add `PasswordHash` and `PasswordSalt` columns to the `Usuarios` table.
  - Keep your existing columns (DNI, name, etc.) intact.

> ✅ Your database will now be ready for secure user registration/login.

De momento no voy a hacer las migraciones esas de actualizar la bd.

De los pasos que me dijo para implement basic authentication, estos son los hechos y los no hechos por mí hasta ahora:

1. Create a “User” model poniendo **pwhash** y **pwsalt** en lugar de raw string pw (esto pasa para DTORegistro y DTOlogin).
2. Add DbSet<Usuario> in your AppDbContext (ya lo tenía). **Run migrations to udpate db.**
3. Create **DTOs for Register/Login**
4. Create a **service for pw hashing + JWT token generation** i.e. create a folder Services with file **AuthService.cs**.

5. **Add JWT configuration** in **appsettings.json** (del backend)
6. **Enable JWT Authentication** in **Program.cs** (del backend)
7. Create an **AuthenticationController**
8. **Protect** your existing **CRUD Controllers**
9. **Test** in **Swagger**

3️⃣ Create a new EF migration

After changing the `Usuario` model:

1. Open a terminal in the `ReservasApi` **folder** (where `ReservasApi.csproj` lives).
2. Run:

```bash
dotnet ef migrations add AddUsuarioAuth
dotnet ef database update
```

- This will:
  - Add `PasswordHash` and `PasswordSalt` columns to the `Usuarios` table.
  - Keep your existing columns (DNI, name, etc.) intact.

> ✅ Your database will now be ready for secure user registration/login.

4️⃣ What comes next

Once the database is updated:

1. Create DTOs for registration and login (so you don't send raw hashes over the API).

Voy a **run migrations** pero siendo consciente de que seguramente me perturba los swagger/postman tests al no cambiar otras cousiñas de mi código.

NEXT PAGE

Before running:
dotnet ef migrations add AddUsuarioAuth
dotnet ef database update

Should I stop backend docker container and sql server docker container ?

No — you **do not need to stop your Docker containers** to run EF Core migrations. In fact, you usually **run migrations against the running database**.

Here's the correct approach:

---

1 Backend container

- You **don't need to stop it**.
- The migrations are applied directly from your local `dotnet ef` command, connecting to the SQL Server in your container.

---

2 SQL Server container

- The container **must be running**, because EF Core needs to connect to the database.
- If it's stopped, migrations will fail because there's no database to connect to.

Pasa que el dotnet ef migrations add AddUsuarioAuth no va porque falla dotnet build:

```
C:\Users\iago.cadavidgonzalez\Proyectos\reservas-app\backend\ReservasApi>dotnet ef migrations AddUsuarioAuth
Build started...
Build failed. Use dotnet build to see the errors.

C:\Users\iago.cadavidgonzalez\Proyectos\reservas-app\backend\ReservasApi>dotnet build
Restauración completada (0,3s)
  ReservasApi error con 1 errores (0,5s)
    C:\Users\iago.cadavidgonzalez\Proyectos\reservas-app\backend\ReservasApi\Program.cs(7,43): error CS0234: El tipo o el nombre del espacio de nombres 'JwtBearer' no exi
ste en el espacio de nombres 'Microsoft.AspNetCore.Authentication' (¿falta alguna referencia de ensamblado?)

Compilación error con 1 errores en 1,4s
```

Option A — Install the compatible NuGet package

- For **.NET 9**, the package you need is `Microsoft.AspNetCore.Authentication.JwtBearer` **version 9.x, not 10.x.**

bash Copy code

```
dotnet add package Microsoft.AspNetCore.Authentication.JwtBearer --version 9.0.0
```

- Then rebuild:

bash Copy code

```
dotnet build
```

✅ This will make the namespace `Microsoft.AspNetCore.Authentication.JwtBearer` available.

```
C:\Users\iago.cadavidgonzalez\Proyectos\reservas-app\backend\ReservasApi>dotnet add package Microsoft.AspNetCore.Authentication.JwtBearer --version 9.0.0

Compilación realizado correctamente en 0,6s
info : La validación de la cadena de certificados X.509 utilizará el almacén de confianza predeterminado seleccionado por .NET para la firma de código.
info : La validación de la cadena de certificados X.509 utilizará el almacén de confianza predeterminado seleccionado por .NET para la marca de tiempo.
info : Agregando PackageReference para el paquete "Microsoft.AspNetCore.Authentication.JwtBearer" al proyecto "C:\Users\iago.cadavidgonzalez\Proyectos\reservas-app\backend\ReservasApi\ReservasApi.csproj".
info : Restaurando paquetes para C:\Users\iago.cadavidgonzalez\Proyectos\reservas-app\backend\ReservasApi\ReservasApi.csproj...
info :   GET https://api.nuget.org/v3-flatcontainer/microsoft.aspnetcore.authentication.jwtbearer/index.json
info :   OK https://api.nuget.org/v3-flatcontainer/microsoft.aspnetcore.authentication.jwtbearer/index.json 387 ms
info :   GET https://api.nuget.org/v3-flatcontainer/microsoft.aspnetcore.authentication.jwtbearer/9.0.0/microsoft.aspnetcore.authentication.jwtbearer.9.0.0.nupkg
info :   OK https://api.nuget.org/v3-flatcontainer/microsoft.aspnetcore.authentication.jwtbearer/9.0.0/microsoft.aspnetcore.authentication.jwtbearer.9.0.0.nupkg 106 ms
info :   GET https://api.nuget.org/v3-flatcontainer/microsoft.identitymodel.protocols.openidconnect/index.json
info :   OK https://api.nuget.org/v3-flatcontainer/microsoft.identitymodel.protocols.openidconnect/index.json 156 ms
info :   GET https://api.nuget.org/v3-flatcontainer/microsoft.identitymodel.protocols.openidconnect/8.0.1/microsoft.identitymodel.protocols.openidconnect.8.0.1.nupkg
info :   OK https://api.nuget.org/v3-flatcontainer/microsoft.identitymodel.protocols.openidconnect/8.0.1/microsoft.identitymodel.protocols.openidconnect.8.0.1.nupkg 51 ms
info :   GET https://api.nuget.org/v3-flatcontainer/microsoft.identitymodel.protocols/index.json
info :   GET https://api.nuget.org/v3-flatcontainer/system.identitymodel.tokens.jwt/index.json
info :   OK https://api.nuget.org/v3-flatcontainer/system.identitymodel.tokens.jwt/index.json 240 ms
info :   GET https://api.nuget.org/v3-flatcontainer/system.identitymodel.tokens.jwt/8.0.1/system.identitymodel.tokens.jwt.8.0.1.nupkg
info :   OK https://api.nuget.org/v3-flatcontainer/system.identitymodel.tokens.jwt/8.0.1/system.identitymodel.tokens.jwt.8.0.1.nupkg 50 ms
info :   OK https://api.nuget.org/v3-flatcontainer/microsoft.identitymodel.protocols/index.json 427 ms
info :   GET https://api.nuget.org/v3-flatcontainer/microsoft.identitymodel.protocols/8.0.1/microsoft.identitymodel.protocols.8.0.1.nupkg
info :   OK https://api.nuget.org/v3-flatcontainer/microsoft.identitymodel.protocols/8.0.1/microsoft.identitymodel.protocols.8.0.1.nupkg 47 ms
info :   GET https://api.nuget.org/v3-flatcontainer/microsoft.identitymodel.tokens/index.json
info :   GET https://api.nuget.org/v3-flatcontainer/microsoft.identitymodel.jsonwebtokens/index.json
info :   OK https://api.nuget.org/v3-flatcontainer/microsoft.identitymodel.tokens/index.json 149 ms
info :   OK https://api.nuget.org/v3-flatcontainer/microsoft.identitymodel.jsonwebtokens/index.json 162 ms
info :   GET https://api.nuget.org/v3-flatcontainer/microsoft.identitymodel.tokens/8.0.1/microsoft.identitymodel.tokens.8.0.1.nupkg
info :   GET https://api.nuget.org/v3-flatcontainer/microsoft.identitymodel.jsonwebtokens/8.0.1/microsoft.identitymodel.jsonwebtokens.8.0.1.nupkg
info :   OK https://api.nuget.org/v3-flatcontainer/microsoft.identitymodel.tokens/8.0.1/microsoft.identitymodel.tokens.8.0.1.nupkg 39 ms
info :   OK https://api.nuget.org/v3-flatcontainer/microsoft.identitymodel.jsonwebtokens/8.0.1/microsoft.identitymodel.jsonwebtokens.8.0.1.nupkg 48 ms
info :   GET https://api.nuget.org/v3-flatcontainer/microsoft.identitymodel.logging/index.json
info :   OK https://api.nuget.org/v3-flatcontainer/microsoft.identitymodel.logging/index.json 176 ms
info :   GET https://api.nuget.org/v3-flatcontainer/microsoft.identitymodel.logging/8.0.1/microsoft.identitymodel.logging.8.0.1.nupkg
info :   OK https://api.nuget.org/v3-flatcontainer/microsoft.identitymodel.logging/8.0.1/microsoft.identitymodel.logging.8.0.1.nupkg 37 ms
info :   GET https://api.nuget.org/v3-flatcontainer/microsoft.identitymodel.abstractions/index.json
info :   OK https://api.nuget.org/v3-flatcontainer/microsoft.identitymodel.abstractions/index.json 168 ms
info :   GET https://api.nuget.org/v3-flatcontainer/microsoft.identitymodel.abstractions/8.0.1/microsoft.identitymodel.abstractions.8.0.1.nupkg
info :   OK https://api.nuget.org/v3-flatcontainer/microsoft.identitymodel.abstractions/8.0.1/microsoft.identitymodel.abstractions.8.0.1.nupkg 41 ms
info : Se ha instalado Microsoft.AspNetCore.Authentication.JwtBearer 9.0.0 de https://api.nuget.org/v3/index.json a C:\Users\iago.cadavidgonzalez\.nuget\packages\microsoft.aspnetcore.authentication.jwtbearer\9.0.0 con el hash de contenido bs+1Pq3vQdS2lTyxNUd9fEhtMsq3eLUpK36k2t56iDMVrk6OrAoFtvrQrTK0Y0OetTcJrUkGU7hBlf+ORzHLqQ==.
info : Se ha instalado Microsoft.IdentityModel.Protocols.OpenIdConnect 8.0.1 de https://api.nuget.org/v3/index.json a C:\Users\iago.cadavidgonzalez\.nuget\packages\microsoft.identitymodel.protocols.openidconnect\8.0.1 con el hash de contenido AQDbfpL+yzuuGhO/mQhKNsp44pm5Jv8/BI4KiFXR7beVGZoSH35zMV3PrmcfvSTsyI6qrcR898NzUauD6SRigg==.
```

Finaliza en next page

```
info : Se ha instalado Microsoft.IdentityModel.Logging 8.0.1 de https://api.nuget.org/v3/index.json a C:\Users\iago.cadavidgonzalez\.nuget\packages\microsoft.identitymode
l.logging\8.0.1 con el hash de contenido UCPF2exZqBXe7v/6sGNiM6zCQOUXXQ9+v5VTb9gPB8ZSUPnX53BxlN78v2jsbIvK9Dq4GovQxo23x8JgWvm/Qg==.
info : Se ha instalado System.IdentityModel.Tokens.Jwt 8.0.1 de https://api.nuget.org/v3/index.json a C:\Users\iago.cadavidgonzalez\.nuget\packages\system.identitymodel.t
okens.jwt\8.0.1 con el hash de contenido GJw3bYkWpOgvN3tJo5X4lYUeIFA2HD293FPUhKmp7qxS+g5ywAb34Dnd3cDAFLkcMohy5XTpoaZ4uAHuw0uSPQ==.
info : Se ha instalado Microsoft.IdentityModel.Abstractions 8.0.1 de https://api.nuget.org/v3/index.json a C:\Users\iago.cadavidgonzalez\.nuget\packages\microsoft.identit
ymodel.abstractions\8.0.1 con el hash de contenido OtlIWcyX01olfdevPKZdIPfBEvbcioDyBiE/Z2lHsopsMD7twcKtlN9kMevHmI5IIPhFpfwCIiR6qHQz1WHUIw==.
info : Se ha instalado Microsoft.IdentityModel.Protocols 8.0.1 de https://api.nuget.org/v3/index.json a C:\Users\iago.cadavidgonzalez\.nuget\packages\microsoft.identitymo
del.protocols\8.0.1 con el hash de contenido uA2vpKqU3I2mBBEaeJAWPTjT9v1TZrGWKdgK6G5qJd03CLx83kdiqO9cmiK8/n1erkHzFBwU/RphP83aAe3i3g==.
info : Se ha instalado Microsoft.IdentityModel.JsonWebTokens 8.0.1 de https://api.nuget.org/v3/index.json a C:\Users\iago.cadavidgonzalez\.nuget\packages\microsoft.identi
tymodel.jsonwebtokens\8.0.1 con el hash de contenido s6++gF9x0rQApQzOBbSyp4jUaAlwm+DroKfL8gdOHxs83k8SJfUXhuc46rDB3rNXBQ1MVRxqKUrqFhO/M0E97g==.
info : Se ha instalado Microsoft.IdentityModel.Tokens 8.0.1 de https://api.nuget.org/v3/index.json a C:\Users\iago.cadavidgonzalez\.nuget\packages\microsoft.identitymodel
.tokens\8.0.1 con el hash de contenido kDimB6Dkd3nkW2oZPDkMkVHfQt3IDqO5gL0oa8WVy3OP4uE8Ij+8TXnqg9TOd9ufjsY3IDiGz7pCUbnfL18tjg==.
info :   GET https://api.nuget.org/v3/vulnerabilities/index.json
info :   OK https://api.nuget.org/v3/vulnerabilities/index.json 46 ms
info :   GET https://api.nuget.org/v3-vulnerabilities/2025.11.19.23.31.42/vulnerability.base.json
info :   GET https://api.nuget.org/v3-vulnerabilities/2025.11.19.23.31.42/2025.11.25.05.32.00/vulnerability.update.json
info :   OK https://api.nuget.org/v3-vulnerabilities/2025.11.19.23.31.42/vulnerability.base.json 45 ms
info :   OK https://api.nuget.org/v3-vulnerabilities/2025.11.19.23.31.42/2025.11.25.05.32.00/vulnerability.update.json 54 ms
info : El paquete "Microsoft.AspNetCore.Authentication.JwtBearer" es compatible con todos los marcos de trabajo especificados del proyecto "C:\Users\iago.cadavidgonzalez\
Proyectos\reservas-app\backend\ReservasApi\ReservasApi.csproj".
info : Se agregó PackageReference para la versión "9.0.0" del paquete "Microsoft.AspNetCore.Authentication.JwtBearer" al archivo "C:\Users\iago.cadavidgonzalez\Proyectos\
reservas-app\backend\ReservasApi\ReservasApi.csproj".
info : Generación de archivo MSBuild C:\Users\iago.cadavidgonzalez\Proyectos\reservas-app\backend\ReservasApi\obj\ReservasApi.csproj.nuget.g.props.
info : Escribiendo el archivo de recursos en el disco. Ruta de acceso: C:\Users\iago.cadavidgonzalez\Proyectos\reservas-app\backend\ReservasApi\obj\project.assets.json
log  : Se ha restaurado C:\Users\iago.cadavidgonzalez\Proyectos\reservas-app\backend\ReservasApi\ReservasApi.csproj (en 4,21 s).

C:\Users\iago.cadavidgonzalez\Proyectos\reservas-app\backend\ReservasApi>
```

Ahora aparece JwtBearer en el csproj:

Program.cs | ReservasApi.csproj | appsettings.json

C: > Users > iago.cadavidgonzalez > Proyectos > reservas-app > backend > ReservasApi > ReservasApi.csproj

```
1   <Project Sdk="Microsoft.NET.Sdk.Web">
2
3     <PropertyGroup>
4       <TargetFramework>net9.0</TargetFramework>
5       <Nullable>enable</Nullable>
6       <ImplicitUsings>enable</ImplicitUsings>
7     </PropertyGroup>
8
9     <ItemGroup>
10      <PackageReference Include="Confluent.Kafka" Version="2.12.0" />
11      <PackageReference Include="Microsoft.AspNetCore.Authentication.JwtBearer" Version="9.0.0" />
12      <PackageReference Include="Microsoft.AspNetCore.OpenApi" Version="9.0.10" />
13      <PackageReference Include="Microsoft.EntityFrameworkCore" Version="9.0.10" />
14      <PackageReference Include="Microsoft.EntityFrameworkCore.Design" Version="9.0.10">
15        <IncludeAssets>runtime; build; native; contentfiles; analyzers; buildtransitive</IncludeAssets>
16        <PrivateAssets>all</PrivateAssets>
17      </PackageReference>
18      <PackageReference Include="Microsoft.EntityFrameworkCore.SqlServer" Version="9.0.10" />
19      <PackageReference Include="Microsoft.EntityFrameworkCore.Tools" Version="9.0.10">
20        <PrivateAssets>all</PrivateAssets>
21        <IncludeAssets>runtime; build; native; contentfiles; analyzers; buildtransitive</IncludeAssets>
22      </PackageReference>
23      <PackageReference Include="Swashbuckle.AspNetCore" Version="9.0.6" />
24    </ItemGroup>
25
26  </Project>
27
```

Para quitar las dos alertas sobre pwhash y pwsalt simplemente pongo byte[]? en lugar de byte[], entonces vuelvo a hacer dotnet build:

```
C:\Users\iago.cadavidgonzalez\Proyectos\reservas-app\backend\ReservasApi>dotnet build
Restauración completada (0,3s)
  ReservasApi error con 1 errores y 1 advertencias (0,7s)
    C:\Users\iago.cadavidgonzalez\Proyectos\reservas-app\backend\ReservasApi\Program.cs(54,28): error CS0246: El nombre del tipo o del espacio de nombres 'AuthService' no se encontró (¿falta una directiva using o una referencia de ensamblado?)
    C:\Users\iago.cadavidgonzalez\Proyectos\reservas-app\backend\ReservasApi\Program.cs(49,40): warning CS8604: Posible argumento de referencia nulo para el parámetro "s" en "byte[] Encoding.GetBytes(string s)".

Compilación error con 1 errores y 1 advertencias en 1,6s

C:\Users\iago.cadavidgonzalez\Proyectos\reservas-app\backend\ReservasApi>
```

2️⃣ Error: `AuthService` not found (CS0246)

```
powershell

error CS0246: El nombre del tipo o del espacio de nombres 'AuthService' no se encontró
```

Why:

- You haven't created the `AuthService` class yet.
- In your `Program.cs` you wrote:

```
csharp

builder.Services.AddScoped<AuthService>();
```

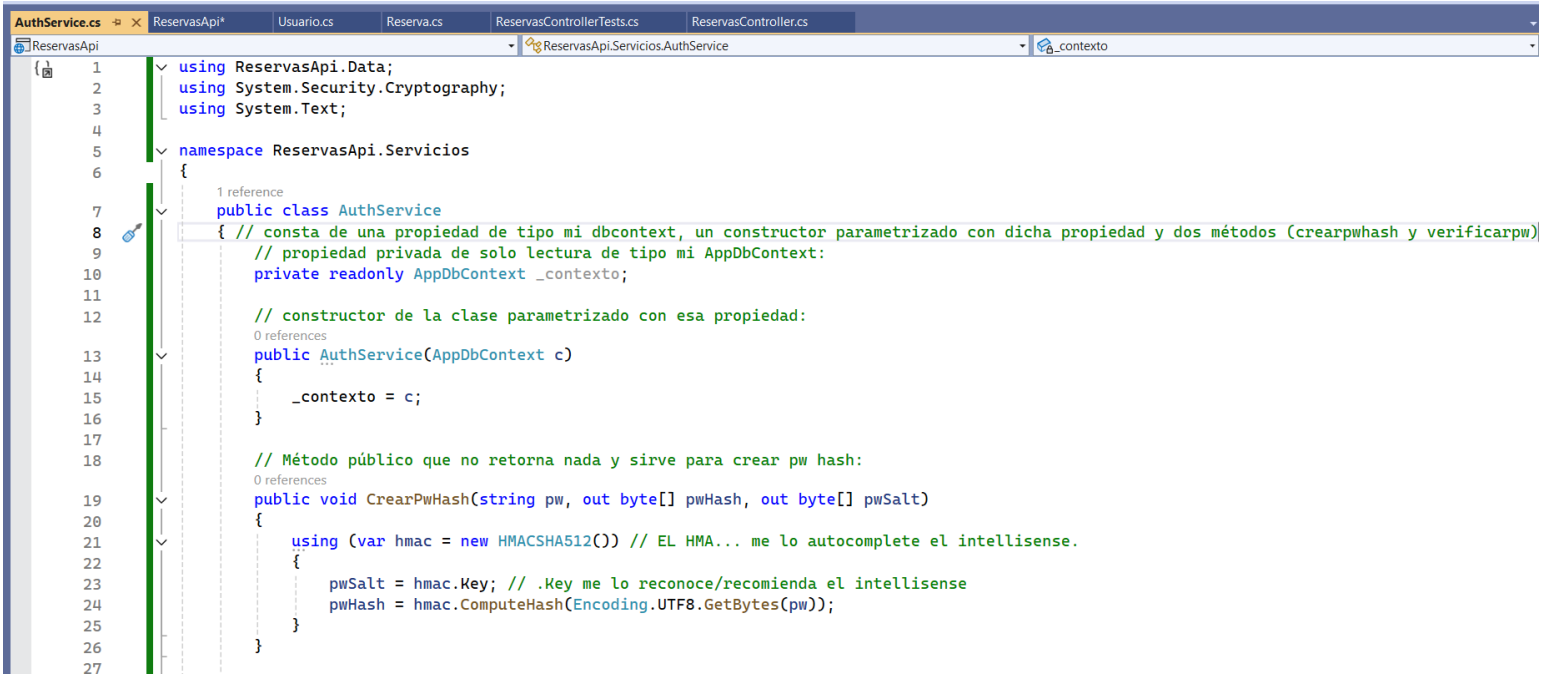
- This fails because **there's no such class defined**.

Fix:

1. Create a new file `Services/AuthService.cs`:

Osease

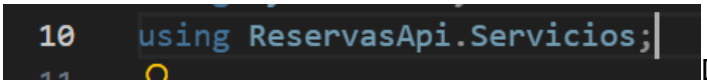
4. Create·a·**service·for·pw·hashing·+·JWT·token·generation**·i.e.·create·a·folder·Services·with·file·AuthService.cs.¶



```
28          // Método que retorna buleano y verifica la password:
            0 references
29          public bool VerificarPwHash(string pw, byte[] hashGuardado, byte[] saltGuardado)
30          {
31              using (var hmac = new HMACSHA512(saltGuardado))
32              {
33                  var hashComputado = hmac.ComputeHash(Encoding.UTF8.GetBytes(pw));
34                  return hashComputado.SequenceEqual(hashGuardado);
35              }
36          } // final del método VerificarPwHash
37
38      } // final de la public class AuthService, la cual uso en Program.cs en línea builder.Services.AddScoped<AuthService>();
39  } // namespace
40
```

Your AuthService implementation is correct, safe and fully functional.

Necesario añadir al comienzo de Program.cs:



De lo contrario del dotnet build seguirá dando el mismo error de que no reconoce AuthService como si todavía no existiese.

Una vez añadio, ese using, vuelvo a hacer dotnet build:

```
C:\Users\iago.cadavidgonzalez\Proyectos\reservas-app\backend\ReservasApi>dotnet build
Restauración completada (0,3s)
  ReservasApi correcto con 1 advertencias (8,1s) → bin\Debug\net9.0\ReservasApi.dll
    C:\Users\iago.cadavidgonzalez\Proyectos\reservas-app\backend\ReservasApi\Program.cs(50,40): warning CS8604: Posible argumento de referencia nulo para el parámetro "s" en "byte[] Encoding.GetBytes(string s)".

Compilación correcto con 1 advertencias en 9,0s
```

“The compiler warning does NOT mean the value is null; it only means the compiler cannot verify that it won’t be null.”

Según chatgpt, si añado

```
?? throw new Exception("Missing Jwt:Key in configuration")
```

En la línea de Program.cs que da la advertencia al correr el dotnet build, entonces me aseguro de que no saltará dicha advertencia y al mismo tiempo como en appsettings.json sí tengo lo que tengo que tener al respecto de la jwt:key, tampoco caeré en esa Exception y por ende no me dará ese mensaje sino que caeré en el caso de la izquierda del ??.

En concreto la cosa queda así:

```
IssuerSigningKey = new SymmetricSecurityKey(
    Encoding.UTF8.GetBytes(
        builder.Configuration["Jwt:Key"]
            ?? throw new Exception("Missing Jwt:Key in configuration")
    )
)
```

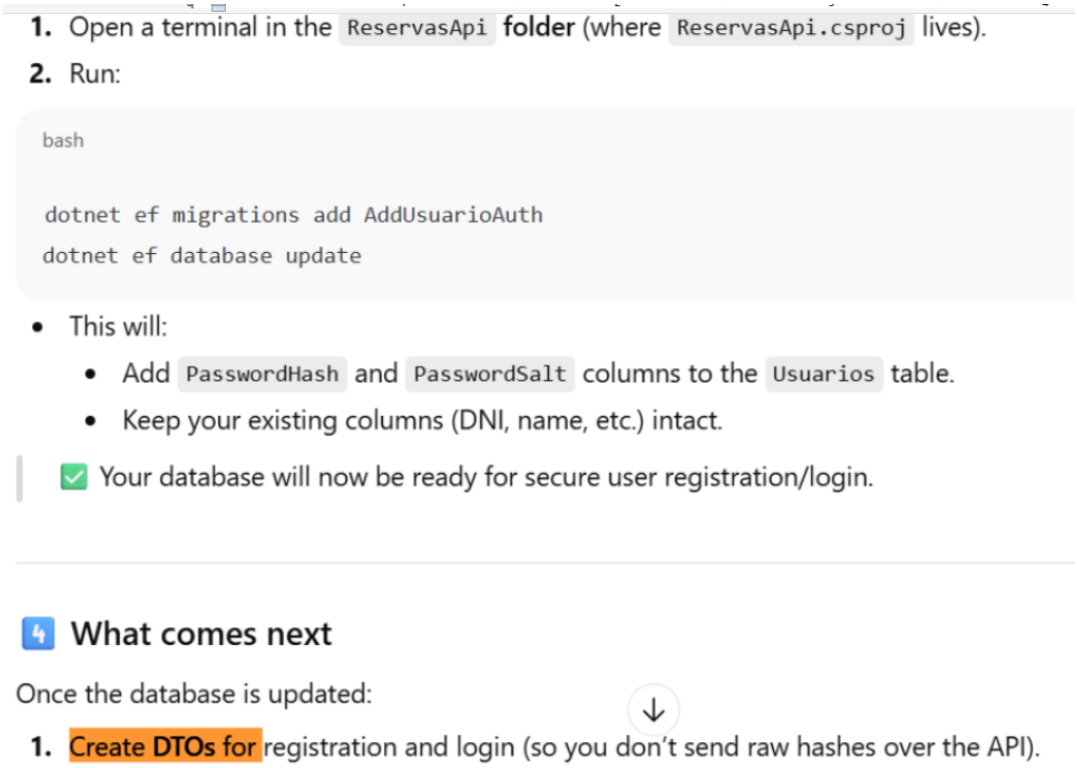
Y entonces:

```
C:\Users\iago.cadavidgonzalez\Proyectos\reservas-app\backend\ReservasApi>dotnet build
Restauración completada (0,3s)
  ReservasApi realizado correctamente (1,2s) → bin\Debug\net9.0\ReservasApi.dll

Compilación realizado correctamente en 2,0s

C:\Users\iago.cadavidgonzalez\Proyectos\reservas-app\backend\ReservasApi>
```

Recordemos que todo esto de implementar el AuthService lo hice porque no podía hacer dotnet build y por ende tampoco el dotnet ef migrations add y por ende no me podía poner con los dtos:



1. Open a terminal in the `ReservasApi` **folder** (where `ReservasApi.csproj` lives).
2. Run:

```
bash

dotnet ef migrations add AddUsuarioAuth
dotnet ef database update
```

- This will:
  - Add `PasswordHash` and `PasswordSalt` columns to the `Usuarios` table.
  - Keep your existing columns (DNI, name, etc.) intact.

> ✅ Your database will now be ready for secure user registration/login.

4️⃣ What comes next

Once the database is updated:

1. Create DTOs for registration and login (so you don't send raw hashes over the API).

Hago ahora el dotnet ef migrations add AddUsuarioAuth (ya que ahora el dotnet build va perfecto)

```
C:\Users\iago.cadavidgonzalez\Proyectos\reservas-app\backend\ReservasApi>dotnet build
Restauración completada (0,3s)
  ReservasApi realizado correctamente (1,2s) → bin\Debug\net9.0\ReservasApi.dll

Compilación realizado correctamente en 2,0s

C:\Users\iago.cadavidgonzalez\Proyectos\reservas-app\backend\ReservasApi>dotnet ef migrations add AddUsuarioAuth
Build started...
Build succeeded.
An operation was scaffolded that may result in the loss of data. Please review the migration for accuracy.
Done. To undo this action, use 'ef migrations remove'

C:\Users\iago.cadavidgonzalez\Proyectos\reservas-app\backend\ReservasApi>
```

```
C:\Users\iago.cadavidgonzalez\Proyectos\reservas-app\backend\ReservasApi>dotnet ef database update
Build started...
Build succeeded.
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (14ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      SELECT 1
info: Microsoft.EntityFrameworkCore.Migrations[20411]
      Acquiring an exclusive lock for migration application. See https://aka.ms/efcore-docs-migrations-lock for more information if this takes too long.
Acquiring an exclusive lock for migration application. See https://aka.ms/efcore-docs-migrations-lock for more information if this takes too long.
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (23ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      DECLARE @result int;
      EXEC @result = sp_getapplock @Resource = '__EFMigrationsLock', @LockOwner = 'Session', @LockMode = 'Exclusive';
      SELECT @result
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (3ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      IF OBJECT_ID(N'[__EFMigrationsHistory]') IS NULL
      BEGIN
          CREATE TABLE [__EFMigrationsHistory] (
              [MigrationId] nvarchar(150) NOT NULL,
              [ProductVersion] nvarchar(32) NOT NULL,
              CONSTRAINT [PK___EFMigrationsHistory] PRIMARY KEY ([MigrationId])
          );
      END;
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (1ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      SELECT 1
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (2ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      SELECT OBJECT_ID(N'[__EFMigrationsHistory]');
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (43ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      SELECT [MigrationId], [ProductVersion]
      FROM [__EFMigrationsHistory]
      ORDER BY [MigrationId];
info: Microsoft.EntityFrameworkCore.Migrations[20402]
      Applying migration '20251125125516_AddUsuarioAuth'.
Applying migration '20251125125516_AddUsuarioAuth'.
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (66ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
```

```
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (66ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      DECLARE @var sysname;
      SELECT @var = [d].[name]
      FROM [sys].[default_constraints] [d]
      INNER JOIN [sys].[columns] [c] ON [d].[parent_column_id] = [c].[column_id] AND [d].[parent_object_id] = [c].[object_id]
      WHERE ([d].[parent_object_id] = OBJECT_ID(N'[Usuarios]') AND [c].[name] = N'Contrasinal');
      IF @var IS NOT NULL EXEC(N'ALTER TABLE [Usuarios] DROP CONSTRAINT [' + @var + '];');
      ALTER TABLE [Usuarios] DROP COLUMN [Contrasinal];
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (21ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      ALTER TABLE [Usuarios] ADD [PasswordHash] varbinary(max) NOT NULL DEFAULT 0x;
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (4ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      ALTER TABLE [Usuarios] ADD [PasswordSalt] varbinary(max) NOT NULL DEFAULT 0x;
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (4ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      INSERT INTO [__EFMigrationsHistory] ([MigrationId], [ProductVersion])
      VALUES (N'20251125125516_AddUsuarioAuth', N'9.0.10');
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (2ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      DECLARE @result int;
      EXEC @result = sp_releaseapplock @Resource = '__EFMigrationsLock', @LockOwner = 'Session';
      SELECT @result
Done.

C:\Users\iago.cadavidgonzalez\Proyectos\reservas-app\backend\ReservasApi>
```

Tuve que cambiar de ‘db’ a ‘localhost,1433’ en appsettings.json del backend para que ese comando no diera un error al final.

EF Core did not create a new database because the existing one (ReservasDb) already exists. Instead, it **updated your existing database** with the new table/columns. Each **migration** is a set of instructions (C# code) that describes how to **update a database schema**.

> ### 4 What comes next
>
> Once the database is updated:
>
> 1. Create DTOs for registration and login (so you don't send raw hashes over the API).

NEXT PAGE

Is "creating the Register and Login endpoints with JWT so your basic authentication is fully functional." the same as "Create DTOs for registration and login (so you don't send raw hashes over the API)" ?

Yes — those two steps are **closely related, but not exactly the same.** Let me explain carefully.

1 Register and Login endpoints with JWT

This refers to **the actual API methods** you expose in a controller:

- `POST /api/auth/register` → creates a new user in the database
- `POST /api/auth/login` → checks credentials and returns a JWT token

These endpoints handle **the logic**: hashing passwords, verifying hashes, generating JWTs, etc.

2 DTOs for registration and login

DTOs = **Data Transfer Objects** — small classes that define exactly what data the API **receives** or **returns**.

Example:

csharp Copy code

```csharp
public class RegisterDto
{
    public string Dni { get; set; } = string.Empty;
    public string NombreCompleto { get; set; } = string.Empty;
    public string Email { get; set; } = string.Empty;
    public string Password { get; set; } = string.Empty; // plain text from client
}

public class LoginDto
{
    public string Dni { get; set; } = string.Empty;
    public string Password { get; set; } = string.Empty; // plain text from client
}
```

Why you need DTOs:

- You **never send raw** `PasswordHash` or `PasswordSalt` **over the API.**
- You **never receive** `PasswordHash` or `PasswordSalt` **directly** from the client.
- DTOs let you send/receive only the fields you care about (like `Password` in plain text), then your `AuthService` hashes it internally.

What I wanna do now is to "**create DTOs for registration and login** (so I don't send raw hashes over the API).

These **DTOs** are gonna be simple **C# classes** that **define exactly what data the client will send to your API**, so you don't expose PasswordHash or PasswordSalt over the network (supongo que se refiere a las propiedades de Usuario.cs).

I already have a folder called 'DTO', but I have it inside 'Modelos' folder, which is inside 'ReservasApi' folder.

That works perfectly — you don't need a separate top-level `DTOs` folder. ✅

You can keep your DTOs inside:

markdown — Copy code

```
ReservasApi/
 └─ Modelos/
      └─ DTO/
```

Just make sure that:

1. The **namespace matches the folder structure**, for example:

csharp — Copy code

```csharp
namespace ReservasApi.Modelos.DTO
{
    public class RegisterDto
    {
        // ...
    }
}
```

2. When you use them in your controller, you add a `using`:

csharp — Copy code

```csharp
using ReservasApi.Modelos.DTO;
```

Does RegisterDto replace UsuarioDto ? La respuesta es no:

NEXT PAGE

Not exactly — `RegisterDto` **does not replace** `UsuarioDto`; they serve **different purposes**.

Here's the distinction:

---

1 `UsuarioDto`

- Represents **user data that you send to clients** (read-only or safe information).
- Typically **does not include passwords**.
- Example: returning user details after registration or in a GET `/users` endpoint.

csharp Copy cod

```csharp
namespace ReservasApi.Modelos.DTO
{
    public class UsuarioDto
    {
        public string Dni { get; set; } = string.Empty;
        public string NombreCompleto { get; set; } = string.Empty;
        // Other fields you want the client to see
    }
}
```

---

2 `RegisterDto`

- Represents **data the client sends to register a new user**.
- Includes the **plain-text password** (which will be hashed server-side).
- Example: POST `/api/auth/register` endpoint.

csharp Copy cod

```csharp
namespace ReservasApi.Modelos.DTO
{
    public class RegisterDto
    {
        public string Dni { get; set; } = string.Empty;
        public string NombreCompleto { get; set; } = string.Empty;
        public string Email { get; set; } = string.Empty;
        public string Telefono { get; set; } = string.Empty;
        public string Domicilio { get; set; } = string.Empty;
        public DateOnly FechaNacimiento { get; set; }

        public string Password { get; set; } = string.Empty; // plain text from client
    }
}
```

RegisterDto.cs:

NEXT PAGE

RegisterDto.cs:

```
namespace ReservasApi.Modelos.DTO
{
    0 references
    public class RegisterDto
    {
        0 references
        public string Dni { get; set; } = string.Empty; // igual que en Usuario.cs
        0 references
        public string NombreCompleto { get; set; } = string.Empty; // igual que en Usuario.cs
        0 references
        public string Email { get; set; } = string.Empty; // igual que en Usuario.cs
        0 references
        public string Telefono { get; set; } = string.Empty; // igual que en Usuario.cs
        0 references
        public string Domicilio { get; set; } = string.Empty; // igual que en Usuario.cs
        0 references
        public DateOnly FechaNacimiento { get; set; } // igual que en Usuario.cs

        0 references
        public string Password { get; set; } = string.Empty; // plain text from client
    } // final de la clase RegisterDto
}
```

I've just written RegisterDto, do I need to write any other Dto for our purpose of implementing basic authentication ?

Yes — for basic authentication you typically need **one more DTO** for login.

Here's why and what it looks like:

1️⃣ `LoginDto`

- Purpose: the client sends **credentials** to log in.
- Only needs **identifier** (DNI in your case) and **password**.

NEXT PAGE

Example:

```csharp
namespace ReservasApi.Modelos.DTO
{
    public class LoginDto
    {
        public string Dni { get; set; } = string.Empty;
        public string Password { get; set; } = string.Empty;
    }
}
```

- You do **not** send `PasswordHash` or `PasswordSalt` from the client.
- The API uses your `AuthService` to **verify the password** and issue a JWT.

The client does not send, diría yo en 1er punto.

So the following is enough for my LoginDto ?

```
namespace ReservasApi.Modelos.DTO
{
   public class LoginDto
   {
      public string Dni { get; set; } = string.Empty;
      public string Password { get; set; } = string.Empty;
   }
}
```

Yes — that is **perfect and sufficient** for your LoginDto. ✅

- It only includes the **fields the client needs to send**.
- It does **not expose** `PasswordHash` **or** `PasswordSalt`, which is exactly what you want.
- The `AuthService` will take care of verifying the password using the hash and salt stored in your database.

With `RegisterDto` and `LoginDto` in place, you now have all the input DTOs needed for basic authentication.

The next step is usually to create the **AuthController** that uses these DTOs along with `AuthService` to handle registration and login and generate JWT tokens.

De los pasos que me dijo para implement basic authentication, estos son los hechos y los no hechos por mí hasta ahora:¶

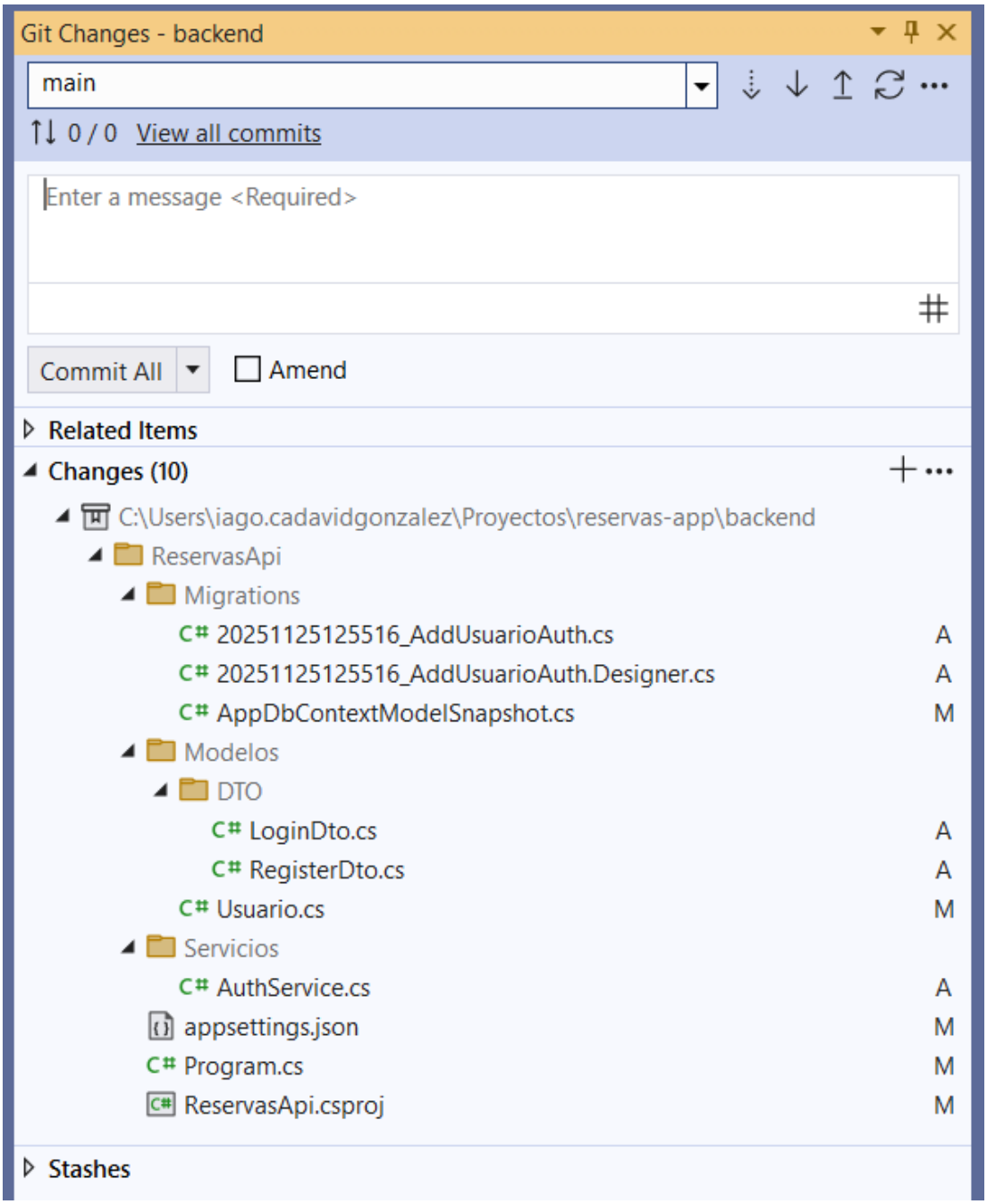
1. Create a "User" model poniendo **pwhash** y **pwsalt** en lugar de raw string pw (esto pasa para DTORegistro y DTOlogin).¶
2. Add DbSet<Usuario> in your AppDbContext (ya lo tenía).. **Run migrations to udpate db.**¶
3. Create **DTOs for Register/Login**¶
4. Create a **service for pw hashing + JWT token generation** i.e. create a folder Services with file **AuthService.cs**.¶

Page Break

5. **Add JWT configuration** in **appsettings.json** (del backend)¶
6. **Enable JWT Authentication** in **Program.cs** (del backend)¶
7. Create an **AuthenticationController**¶
8. **Protect** your existing **CRUD Controllers**¶
9. **Test** in **Swagger**¶

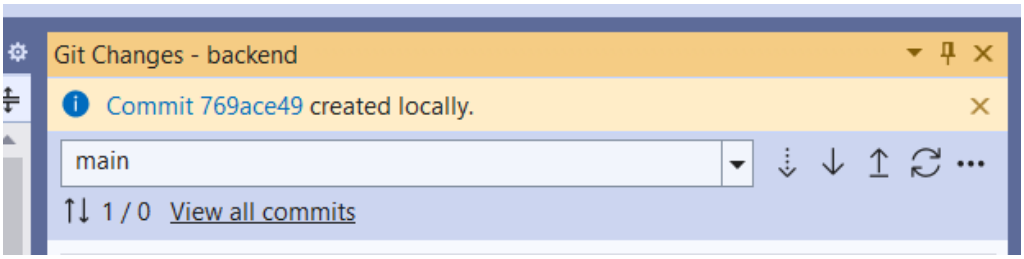
Cambios hasta ahora que voy comitear y pushear: NEXT PAGE

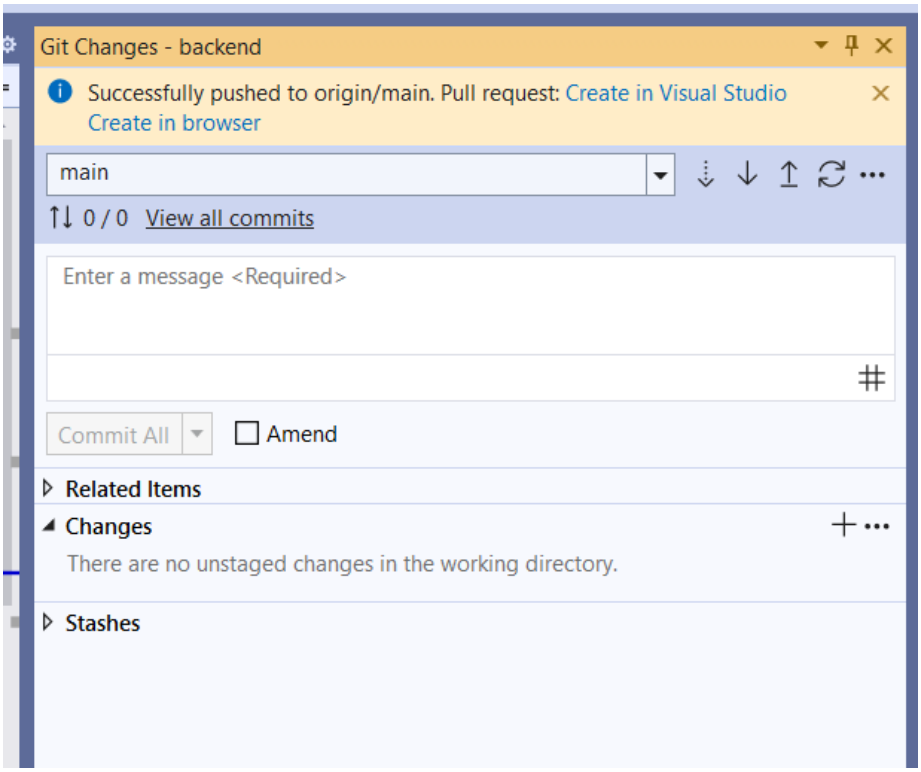
Cambios hasta ahora que voy comitear y pushear:



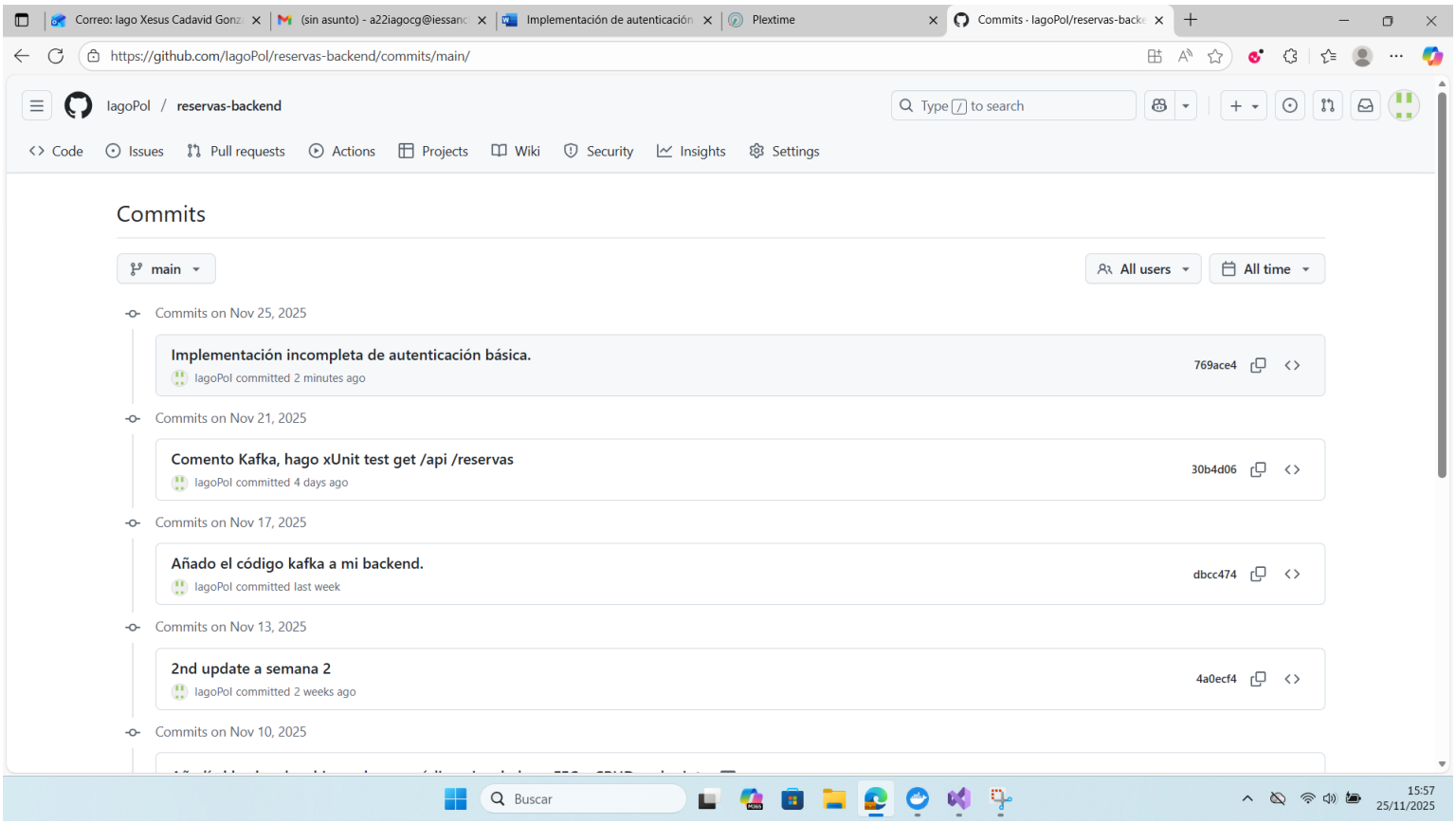
Puse como mensaje pal commit all “implementación incompleta de autenticación básica.”

NEXT PAGE


Git Changes - backend
Commit 769ace49 created locally.
main
1 / 0
View all commits


Git Changes - backend
Successfully pushed to origin/main. Pull request: Create in Visual Studio
Create in browser
main
0 / 0
View all commits
Enter a message <Required>
Commit All
Amend
Related Items
Changes
There are no unstaged changes in the working directory.
Stashes

Next page


Correo: Iago Xesus Cadavid Gonz
(sin asunto) - a22iagocg@iessanc
Implementación de autenticación
Plextime
Commits · IagoPol/reservas-backe
https://github.com/IagoPol/reservas-backend/commits/main/
IagoPol / reservas-backend
Type / to search
Code
Issues
Pull requests
Actions
Projects
Wiki
Security
Insights
Settings
Commits
main
All users
All time
Commits on Nov 25, 2025
Implementación incompleta de autenticación básica.
IagoPol committed 2 minutes ago
769ace4
Commits on Nov 21, 2025
Comento Kafka, hago xUnit test get /api /reservas
IagoPol committed 4 days ago
30b4d06
Commits on Nov 17, 2025
Añado el código kafka a mi backend.
IagoPol committed last week
dbcc474
Commits on Nov 13, 2025
2nd update a semana 2
IagoPol committed 2 weeks ago
4a0ecf4
Commits on Nov 10, 2025
Buscar
15:57
25/11/2025

F

De los pasos que me dijo para implement basic authentication, estos son los hechos y los no hechos por mí hasta ahora:¶

1. Create a "User" model poniendo **pwhash** y **pwsalt** en lugar de raw string pw (esto pasa para DTORegistro y DTOlogin).¶
2. Add DbSet<Usuario> in your AppDbContext (ya lo tenía). **Run migrations to udpate db.**¶
3. Create **DTOs for Register/Login**¶
4. Create a **service for pw hashing + JWT token generation** i.e. create a folder Services with file **AuthService.cs**.¶

Page Break

5. **Add JWT configuration** in **appsettings.json** (del backend)¶
6. **Enable JWT Authentication** in **Program.cs** (del backend)¶
7. Create an **AuthenticationController**¶
8. **Protect** your existing **CRUD Controllers**¶
9. **Test** in **Swagger**¶

"The next step is usually to **create the AuthController** that uses the Register/Login DTOs along with AuthService to handle registration and login and generate JWT tokens."

Please teach me how to create the AuthController.

Great — you already have:

- `RegisterDto`
- `LoginDto`
- `AuthService`
- JWT configured
- `Usuario` model ready
- DB updated

Now we create the **AuthController** that exposes:

- `POST /api/auth/register`
- `POST /api/auth/login`

This (auth) controller will:

1. Receive DTOs
2. Hash passwords (Register)
3. Verify passwords (Login)
4. Generate a JWT token
5. Return safe user info + token

Dentro de ReservasApi/Controllers **creo** ahora **the AuthController file**:

Regarding:

private readonly AppDbContext _context;
private readonly AuthService _authService;
private readonly IConfiguration _config;

Where does IConfiguration come from? I mean I know I created AppDbContext and AuthService .cs files and wrote code into them, but I wouldn't say the same of IConfiguration. Plus, it doesn't seem to correspond to any "using" in the current file AuthController.cs.

Great questions — and everything you're noticing is **correct and expected**.
Let's break it down clearly.

✅ 1. Where does `IConfiguration` come from?

`IConfiguration` is **built into ASP.NET Core**.
It is a framework interface provided automatically by the dependency injection system.

You never create a file for it — it's already available.

It represents your full configuration system:

- `appsettings.json`
- environment variables
- user secrets (if any)
- command-line args
- etc.

F

[HttpPost("register")]
public async Task<IActionResult> Register(RegisterDto dto)

To translate register to Spanish would you say registro (the noun) or registrar (the verb) ?

For API route naming, **always use the verb**, because the endpoint represents an **action**, not an object.

So the correct Spanish translation for:

csharp Copy code

```
[HttpPost("register")]
```

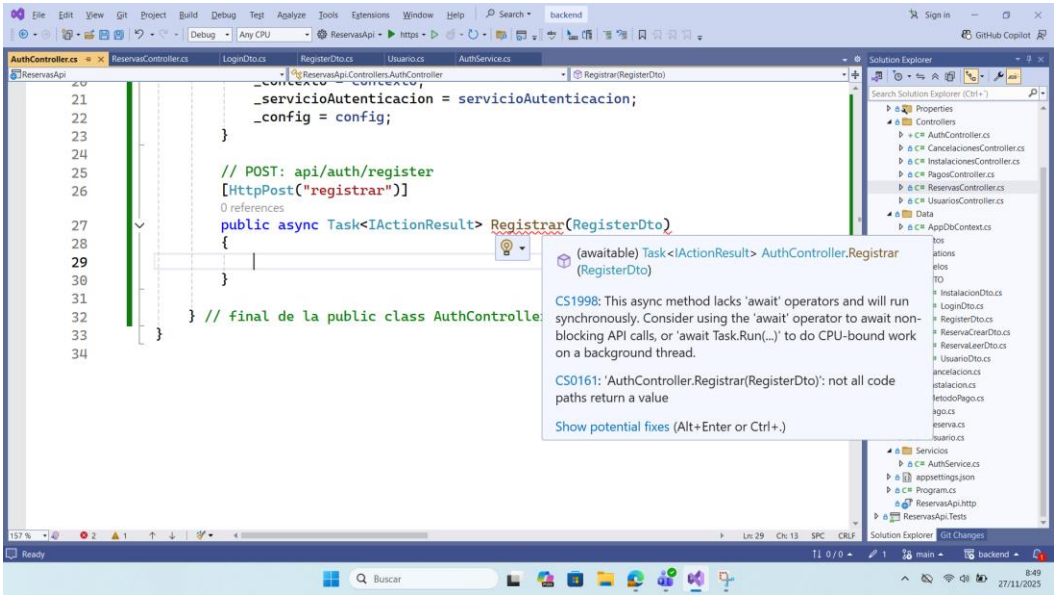
is:

csharp Copy code

```
[HttpPost("registrar")]
```

✔ Why "registrar" is correct

- It matches the HTTP action (POST = perform an action).
- It follows REST conventions: `/login`, `/logout`, `/register` → verbs.
- Spanish verbs are normally used for auth:
  - **registrar** (to register)
  - **iniciar-sesion** or **login**
  - **cerrar-sesion**



⭐ Explanation of what this controller provides

1. Register endpoint (`POST /api/auth/register`)

- Checks if DNI already exists
- Hashes password using your AuthService
- Stores user with hash+salt
- Saves to DB

2. Login endpoint (`POST /api/auth/login`)

- Looks up user
- Verifies password using AuthService
- Generates JWT token
- Returns user fields (safe) + token

3. Protected JWT Token Generator

Uses the secret stored in `appsettings.json`:

```json
"Jwt": {
  "Key": "aP9d73D1jD0m3Ok1Lma17uiWe05vNfg57"
}
```

🚀 Next Step After This

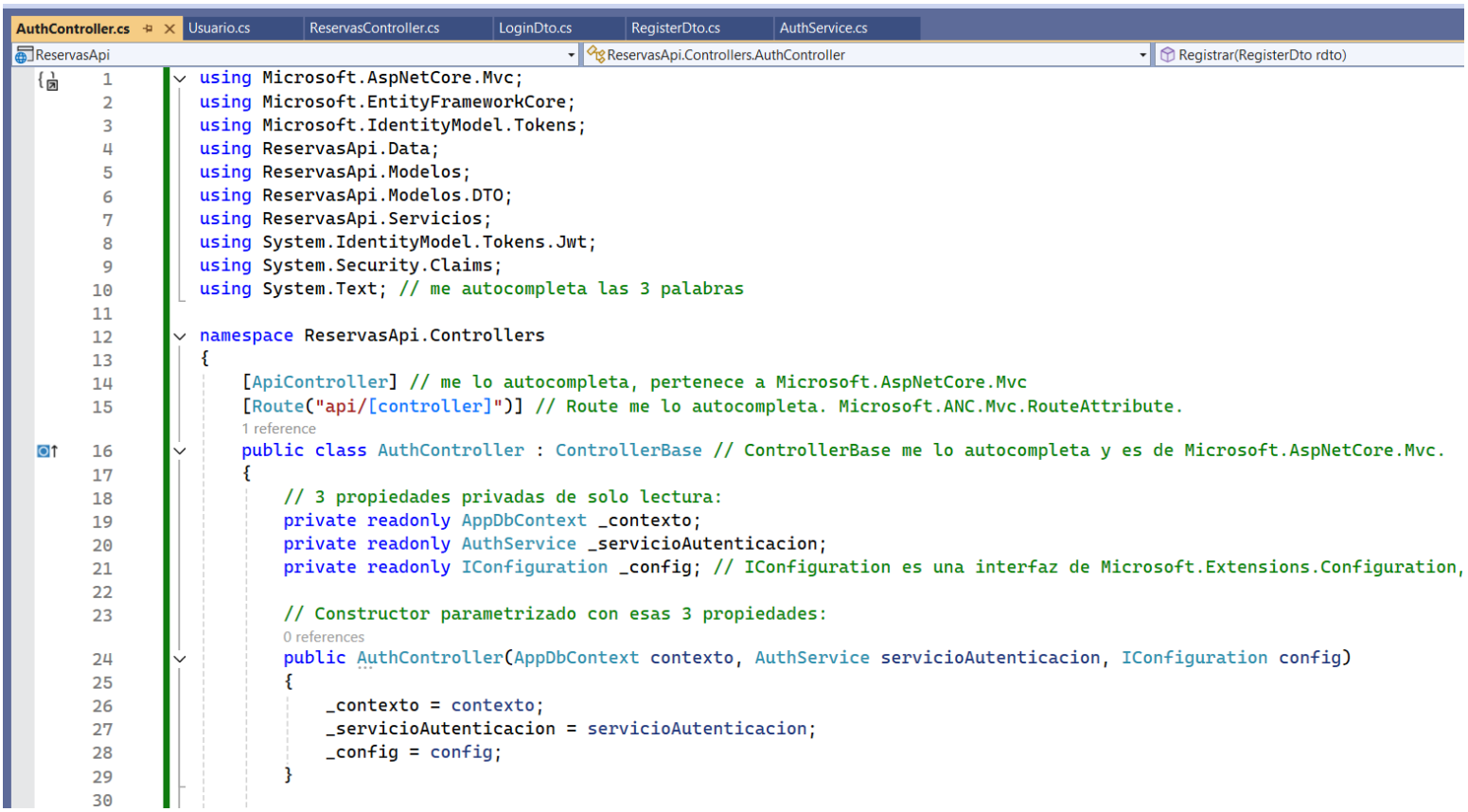
After adding AuthController:

1. Run:

   ```nginx
   dotnet build
   ```

   to confirm no errors.
2. Start your backend
3. Open Swagger
4. Test:
   - **POST /api/auth/register**
   - **POST /api/auth/login**
5. Once login works, you can protect other endpoints using:

   ```csharp
   [Authorize]
   ```




```
// POST: api/auth/register
[HttpPost("registrar")]
0 references
public async Task<IActionResult> Registrar(RegisterDto rdto)
{
    // 1. Comprobar si el usuario ya existe (check if user already exists):
    // AnyAsync asíncronamente determina whether any element of a sequence satisfies a condition.
    var usuarioExiste = await _contexto.Usuarios.AnyAsync(u => u.Dni == rdto.Dni);
    if (usuarioExiste)
        return BadRequest("Ya existe un usuario con este DNI");

    // 2. Crear pw hash + salt con la función que para ello definimos en AuthService.cs:
    _servicioAutenticacion.CrearPwHash(rdto.Password, out byte[] hash, out byte[] salt);

    // 3. Create new Usuario entity i.e. crear un nuevo objeto de la clase Usuario.cs:
    var usuario = new Usuario
    {
        Dni = rdto.Dni,
        NombreCompleto = rdto.NombreCompleto,
        Email = rdto.Email,
        Telefono = rdto.Telefono,
        Domicilio = rdto.Domicilio,
        // fechas:
        FechaNacimiento = rdto.FechaNacimiento,
        FechaRegistro = DateTime.UtcNow, // returns an object whose value is the current UTC date and time.
        // hash y salt serán los recién creados con CrearPwHash.
        PasswordHash = hash,
        PasswordSalt = salt
    };

    // 4. Save to DB:
    _contexto.Usuarios.Add(usuario);
    await _contexto.SaveChangesAsync();

    // 5. retornamos el éxito de la operación de registro de usuario:
    return Ok("Usuario registrado exitósamente");
```

Falta llave de cierre del body

```
// POST: api/auth/login
0 references
public async Task<IActionResult> Loguear(LoginDto ldto)
{
    // 1. Get user from DB:
    var usuario = await _contexto.Usuarios.FirstOrDefaultAsync(u => u.Dni == ldto.Dni);
    if (usuario == null)
        return Unauthorized("DNI y/o contraseña inválidos");

    // 2. Verificar contraseña:
    // en AuthService.cs: public bool VerificarPwHash(string pw, byte[] hashGuardado, byte[] saltGuardado)
    bool esValida = _servicioAutenticacion.VerificarPwHash(ldto.Password, usuario.PasswordHash, usuario.PasswordSalt); //
    if (!esValida) // si la pw NO es válida, entonces:
        return Unauthorized("DNI y/o contraseña inválidos"); // security best practice: never reveal which field failed.

    // 3. Generar el token JWT:
    string tokenCreado = CrearToken(usuario); // CrearToken se define aquí mismo i.e. en AuthController.cs

    return Ok(new
    {
        mensaje = "Login exitoso", // 'mensaje' does not come from anywhere -you invent it on the spot. It's simply a pr
        token = tokenCreado,
        usuario = new
        {
            usuario.Dni,
            usuario.NombreCompleto,
            usuario.Email,
            usuario.Telefono,
            usuario.Domicilio,
            usuario.FechaNacimiento,
            usuario.FechaRegistro
        }
    });
} // final de la función Loguear
```

Finaliza en next page

```
101
102         // HELPER: para Crear JWT Token, definimos función privada con param Usuario y que retorna string:
            1 reference
103         private string CrearToken(Usuario user)
104         {
105             var listaDeClaims = new List<Claim>
106             {
107                 new Claim(ClaimTypes.NameIdentifier, user.Dni),
108                 new Claim(ClaimTypes.Name,user.NombreCompleto)
109             };
110
111             var llave = new SymmetricSecurityKey(
112                 Encoding.UTF8.GetBytes(_config["Jwt:Key"]) // alerta: '_config' is not null here.
113             );
114
115             var credenciales = new SigningCredentials(llave, SecurityAlgorithms.HmacSha512);
116
117             var nuevoTokenJwt = new JwtSecurityToken(
118                 claims: listaDeClaims,
119                 expires: DateTime.UtcNow.AddHours(3),
120                 signingCredentials: credenciales
121             );
122
123             return new JwtSecurityTokenHandler().WriteToken(nuevoTokenJwt);
124         } // final de la función CrearToken.
125
126     } // final de la public class AuthController
127 }
128
```

F

When I test POST /api/auth/register, will this create a new user in table of users ?

Yes — **if your `POST /api/auth/register` endpoint is implemented correctly**, then calling it will **insert a new user into your `Usuarios` table** in SQL Server.

NEXT PAGE

"Yes — if your POST /api/auth/register endpoint is implemented correctly, then calling it will insert a new user into your Usuarios table in SQL Server." then I guess my POST endpoint in UsuariosController is useless, look at it:

```
// POST: api/usuarios
[HttpPost]
public async Task<ActionResult<Usuario>> CrearUsuario(Usuario nuevouser)
{
   _contexto.Usuarios.Add(nuevouser);
   await _contexto.SaveChangesAsync();

   // CreatedAtAction automatically returns '201 Created' + Location header:
   return CreatedAtAction(nameof(GetUsuario), new { id = nuevouser.Dni }, nuevouser);
} // método GetUsuario is defined afterwards in this same UsuariosController.cs file
```

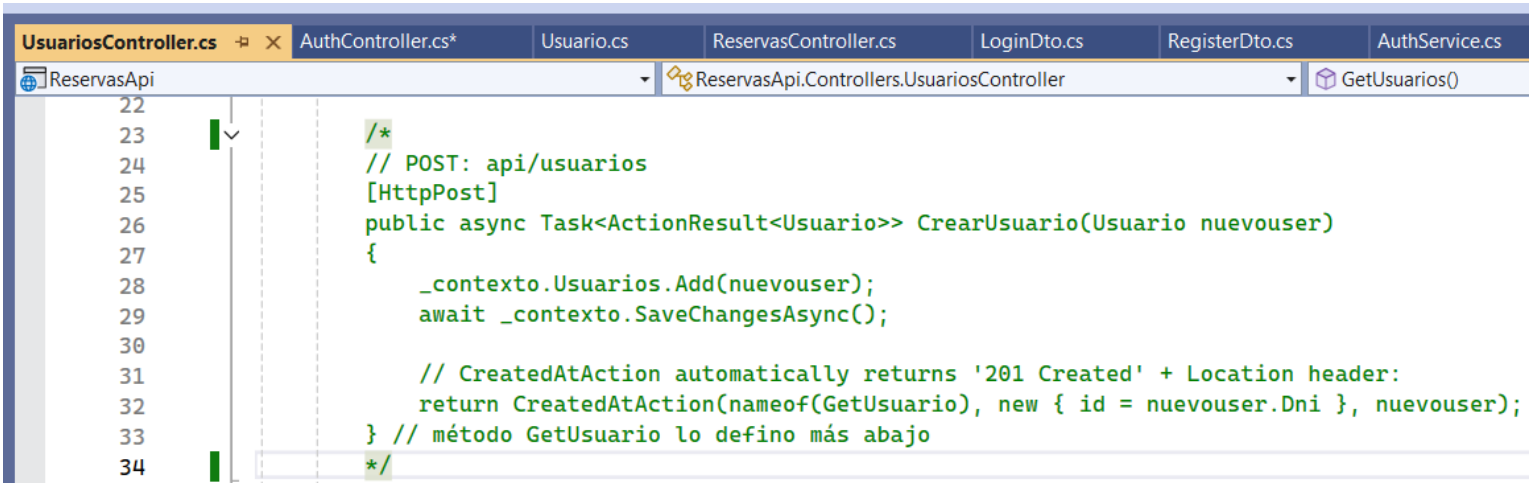
You're **100% correct** — once you implement **POST /api/auth/register**, your existing:

bash Copy code

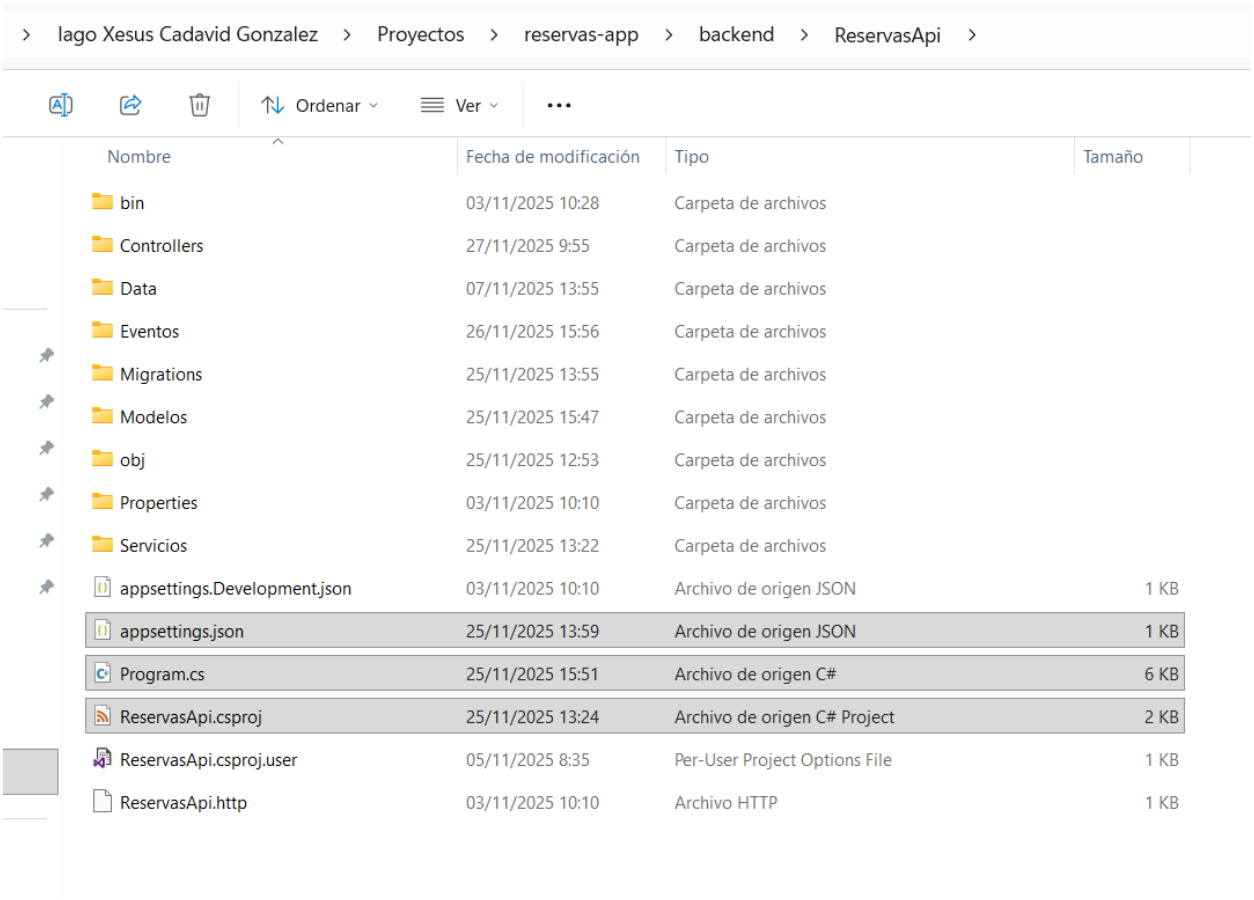
```
POST /api/usuarios
```

endpoint becomes **dangerous** and **should NOT be used anymore for creating users.**

Así que lo comento:

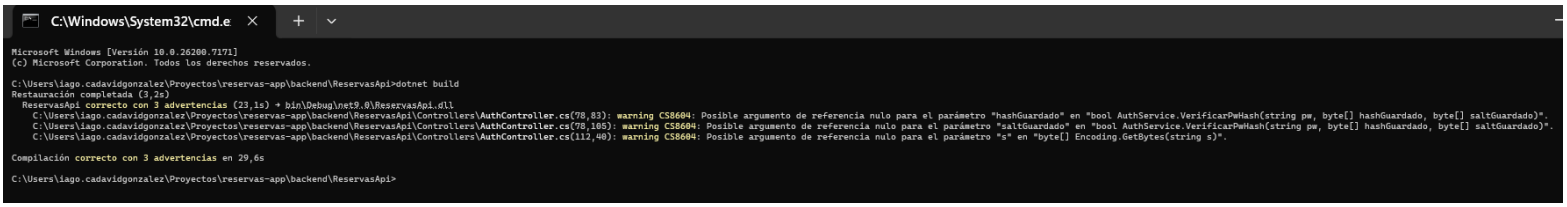


Ahora toca hacer un dotnet build desde carpeta ReservasApi, por ser padre de:

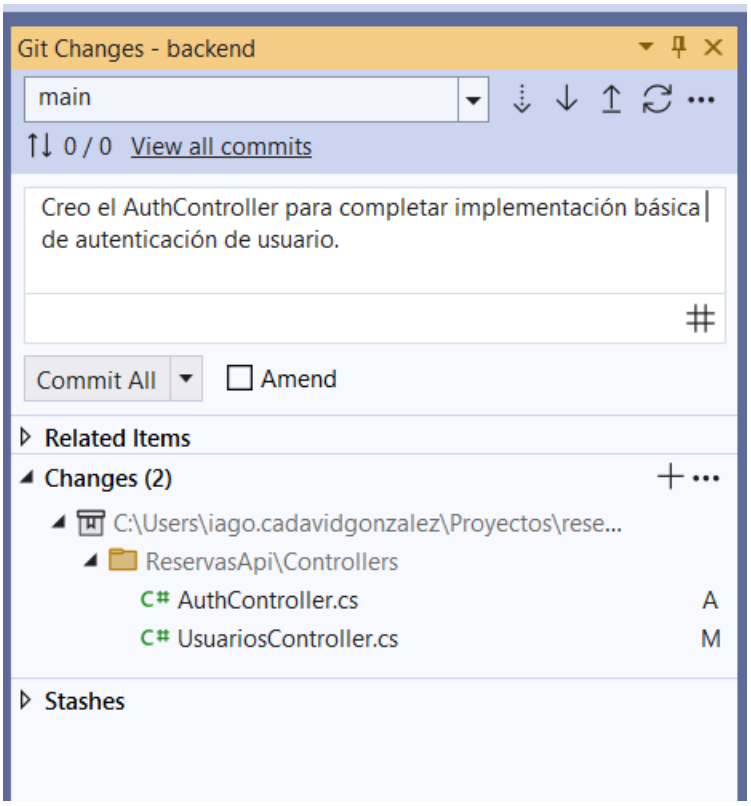


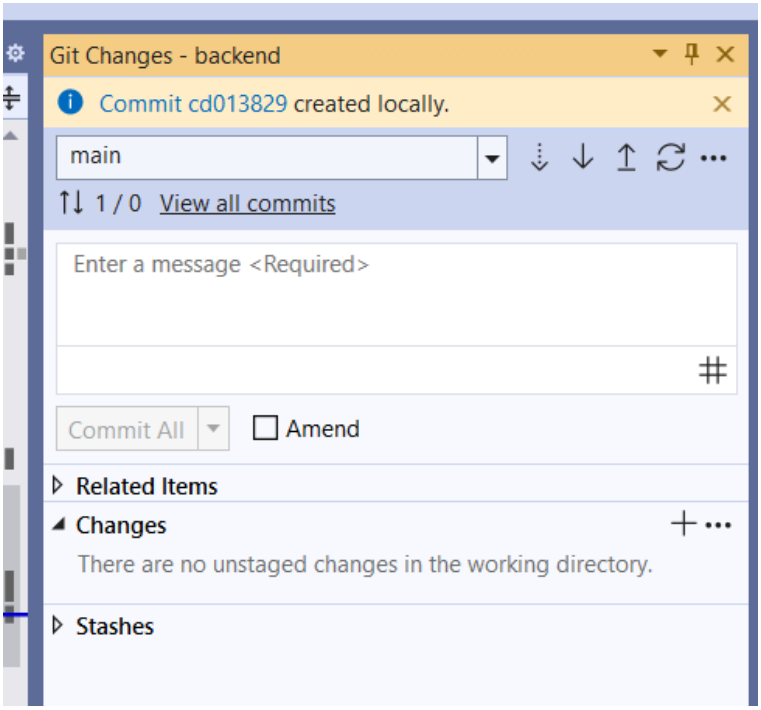
En 1io lo de kafka lo tengo todo comentado y ahora tmb el post de usuarioscontroller.

Build exitoso con 3 alertas que creo que no son relevantes:



NEXT PAGE


Git Changes - backend
main
0 / 0
View all commits
Creo el AuthController para completar implementación básica de autenticación de usuario.
Commit All
Amend
Related Items
Changes (2)
C:\Users\iago.cadavidgonzalez\Proyectos\rese...
ReservasApi\Controllers
AuthController.cs
A
UsuariosController.cs
M
Stashes


Git Changes - backend
Commit cd013829 created locally.
main
1 / 0
View all commits
Enter a message <Required>
Commit All
Amend
Related Items
Changes
There are no unstaged changes in the working directory.
Stashes

does commit include stage

AI Mode All Images Videos News Short videos Forums More Tools

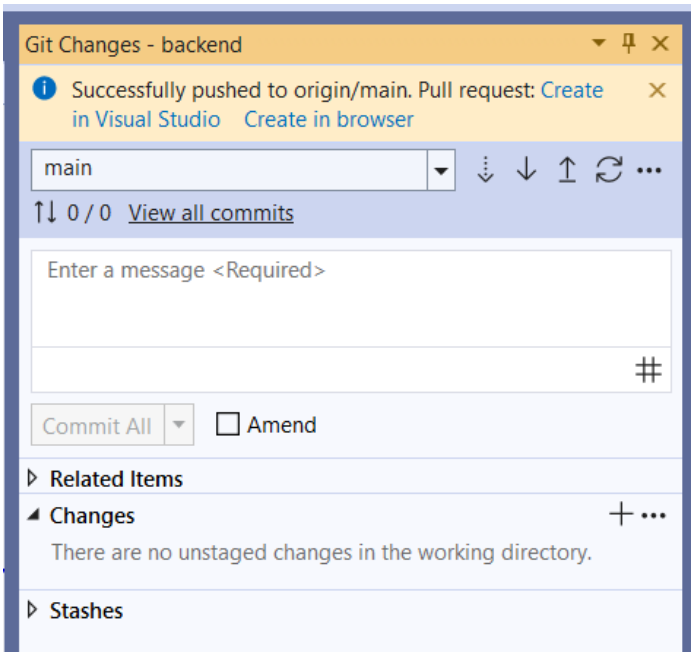
AI Overview

No, a `commit` does not inherently include "stage" because staging is a separate, prior step. A `git commit` command takes a snapshot of the changes that have been previously staged using `git add`. However, you can use the `git commit -a` command as a shortcut to both stage and commit all modified and deleted files in one step.

Standard workflow

- **`git add`**: This command moves changes from your working directory to the staging area, which is like a draft for your next commit.
- **`git commit`**: This command permanently saves the snapshot of all the files currently in the staging area to the repository's history.

El “commit all” del Git Changes de **Visual Studio IDE** me da que es como un git commit –a osease me da que hace both stage and commit, pienso esto porque debido a ese **commit all** paso a ver “**there are no unstaged changes in the working directory**”, cosa que pre-commit all no ponía, por ende pre-commit all sí había unstaged changes, el commit all los stageo (y comiteó -”commit created locally”-).



Sorry, I made a crucial mistake in my previous reply, lemme rewrite it (I capitalize now the correct word that previously was wrong):

Reading "or even push them to the remote repository" makes me realize that actually when Visual Studio IDE showed me the message "Pull request: Create in Visual Studio" was not right after "Commit all", but instead right after clicking on "PUSH" which I did after "Commit all"

Ah! That clears it up perfectly — thanks for the correction. Now the sequence makes total sense. Let's go step by step:

---

1. What actually happened

1. **Commit All** → saved your changes **locally** on your branch.
   - Nothing has gone to the remote repository yet.
2. **Push** → sends your local commits to the **remote repository**.
   - After pushing, Visual Studio saw that your commits are now on a remote branch, but if this branch hasn't been merged into the main branch yet, it assumes your next logical step might be to **request merging**.
   - Hence, it suggested: **"Pull request: Create in Visual Studio."**

2. Why it suggests a Pull Request after a Push

When you push to a branch (especially a feature branch), VS assumes you're working in a collaborative workflow where the usual next step is to:

- Open a **Pull Request** to merge your feature branch into the main branch.
- Let teammates review the changes and approve them.

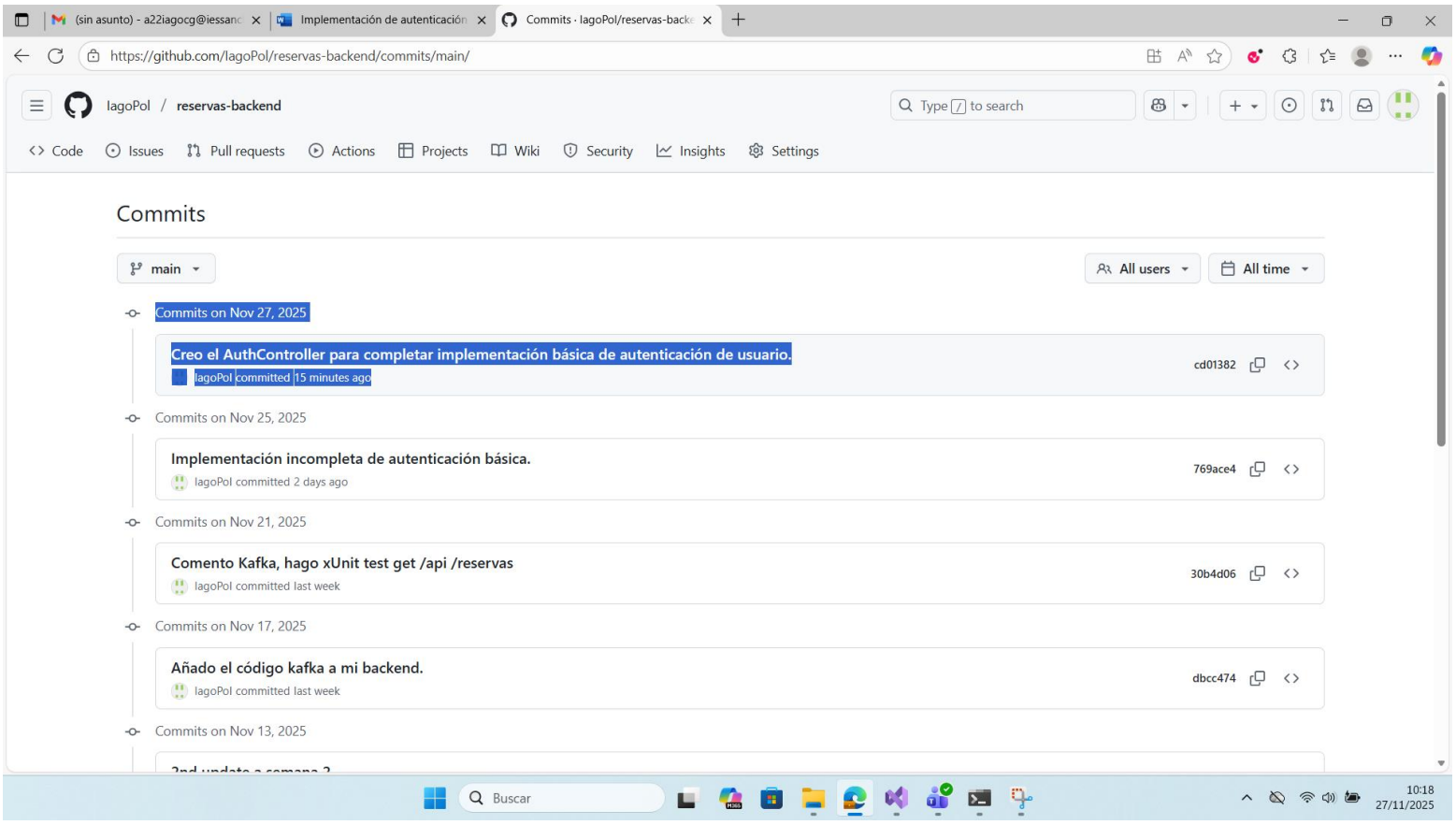
Even if you're working alone, VS still offers this as a convenience because it's the standard workflow in teams.

---

3. Quick workflow summary

Action	What it does	When to do it
Commit	Save changes locally	Whenever you finish a chunk of work
Push	Send commits to remote repository	When you want to share or backup
Pull	Get remote changes into your local branch	Before or after pushing to avoid conflicts
Pull Request	Ask to merge your branch into another	Typically for team collaboration

Lo del when en pull no me queda claro.


(sin asunto) - a22iagocg@iessanc
Implementación de autenticación
Commits · IagoPol/reservas-backe
https://github.com/IagoPol/reservas-backend/commits/main/
IagoPol / reservas-backend
Type / to search
Code
Issues
Pull requests
Actions
Projects
Wiki
Security
Insights
Settings
Commits
main
All users
All time
Commits on Nov 27, 2025
Creo el AuthController para completar implementación básica de autenticación de usuario.
IagoPol committed 15 minutes ago
cd01382
Commits on Nov 25, 2025
Implementación incompleta de autenticación básica.
IagoPol committed 2 days ago
769ace4
Commits on Nov 21, 2025
Comento Kafka, hago xUnit test get /api /reservas
IagoPol committed last week
30b4d06
Commits on Nov 17, 2025
Añado el código kafka a mi backend.
IagoPol committed last week
dbcc474
Commits on Nov 13, 2025
Buscar
10:18
27/11/2025